

Hochschule für angewandte Wissenschaften Hamburg
Automatisierungstechnik,
Verbundprojekt

Geiler Robi macht noch geilere Stunts

Stefo Steffansen, Bent Benjo, Lucy Lukas

Hamburg
4. Juli 2026

Name Stefo Steffansen, Bent Benjo, Lucy Lukas
Matrikelnummer 1111111, 22222222 , 333333333

Hochschule HAW Hamburg
Fakultät Automatisierungstechnik
Studiengang Automatisierungstechnik

Erstprüfer Prof. Dr.-Ing. Maaßi boiiii
Zweitprüfer Prof. Dr. KA

Bearbeitungszeitraum Wann genau 2025/2026

Inhaltsverzeichnis

1	Einleitung	1
2	Konzept	3
2.1	Blockschaltbild	4
2.2	Schnittstellen	4
2.3	Interne State machine der Regelung	5
3	Beobachter	7
3.1	Zielsetzung	7
3.2	Latenzmessung	7
3.3	Kinematisches Beobachtermodell	12
3.4	Beobachterstruktur	14
3.5	Modell Validierung	16
3.6	Beobachterverstärkung Parameterisierung	18
4	Regler	23
4.1	Drehzahlregler	23
4.1.1	Ein- und Ausgangssignale	24
4.1.2	Umrechnung von Roboter- in Motorkoordinaten	25
4.1.3	Interne Reglerstruktur	26
4.2	Positionsregler	27
4.2.1	Ein- und Ausgangssignale	28
4.2.2	Interne Reglerstruktur	29
4.2.3	Umrechnung von Welt- und Roboterkoordinaten	29
4.2.4	Fahralgorithmus mit Gain-Scheduling	31
4.3	Auswertung	35
4.3.1	Vektorfahrweise	35
4.3.2	Positionsfahrweise	37
4.4	Entwurf eines Fuzzy-Reglers	39
4.4.1	Grundlegende Idee und Begründung	39
4.5	Beschreibung Fuzzy-Regler für omnidirektionalen Roboter	40
4.5.1	Erklärung Fuzzy	40
4.5.2	Reglerstruktur für x, y und theta	41
4.5.3	Regelbasis	41
4.5.4	Skalierung der realen Größen	42
4.6	Parameterbestimmung durch Python-Simulation	43
4.6.1	Zentrale Konfiguration	43

4.6.2	Skalierung	44
4.6.3	Simulationsmodell und Ablauf	45
4.6.4	Kostenfunktion	45
4.6.5	Optimierungsverfahren	46
4.6.6	Modellannahmen und Einschränkungen	46
4.6.7	Ergebnis der Simulation	46
4.6.8	Numerische Ergebnisse	47
4.6.9	Interpretation der optimierten Skalierung	48
4.6.10	Bewertung der Simulation	48
4.7	Umsetzung des Fuzzy-Reglers in der Robotersteuerung	49
4.7.1	Integration in die bestehende Steuerungsstruktur	49
4.7.2	Praktische Erweiterungen	49
4.8	Test am realen System, Ergebnisse und Erkenntnisse	50
4.8.1	Positionsfahrt im realen System	50
4.8.2	Anfahrt des Ablagepunkts	52
4.9	Vergleich zwischen Simulation und realem System	54
4.10	Fazit und Ausblick	55

5 Fazit

57

Konventionen

Senkrechte Schrift für Zahlen, Einheiten und Operatoren. Kursive Schrift für mathematische Variablen und physikalische Größen. Ableitungen werden mit einem einfachen Punkt – $\dot{x}$ – über ihnen gekennzeichnet. Der Mittelwert einer Variable wird mit einem Balken über der Variable – $\bar{x}$ – ausgedrückt.

Größe:	Größensymbol:	Einheitenzeichen:
Radgeschwindigkeit	$V_{w1}/V_{w2}/V_{w3}$	m s^{-1}
Geschwindigkeit in Radkoordinaten	v^{wheel}	m s^{-1}
Geschwindigkeit in Roboterkoordinaten	v^{body}	m s^{-1}
Geschwindigkeit in Weltkoordinaten	v^{world}	m s^{-1}
Winkelgeschwindigkeit Roboter	$\dot{\varphi}$	rads^{-1}
Winkelausrichtung Roboter	φ	rad
Roboterradius	R	m

Tabelle 1: Konventionen

1 Einleitung

- Gesamtaufgabe
- Aufgabe unseres Gewerks
-

2 Konzept

Das Konzept zur Erfüllung der Aufgaben von Gewerk 3 wurden in enger Zusammenarbeit mit Gewerk 2 erstellt. Schon in den ersten Wochen des Projektes hat sich Gewerk 2 für ein Gradientenverfahren entschieden, um die Bahnplanung und die Kollisionvermeidung zu realisieren. Als Reaktion auf dieses Vorhaben wurden dann konzeptionell zwei Ansätze verfolgt:

Für die freie Fahrt - also alle Bereiche außerhalb der Taschenbereiche - wird von Gewerk 2 ein Geschwindigkeitssollwert und eine Richtung übermittelt. Dieser Sollwert muss von einem Geschwindigkeitsregler umgesetzt werden. Dieser Modus heißt Vektorregelung. Ein Positionsregler wird in diesem Modus nicht benötigt.

Für die Taschenbereiche - also dort wo hohe Präzision verlangt wird- wird von Gewerk 2 ein Positionssollwert übermittelt. Dieser Sollwert muss von einem Positionsregler umgesetzt werden. Dieser Positionsregler generiert basierend auf der aktuellen Position ein Geschwindigkeitssollwert. Dieser wird dann an den Geschwindigkeitsregler weitergeleitet. Dieser Modus heißt Positionregelung.

In beiden Modi wird die aktuelle Position der Roboter durch einen Beobachter geschätzt. Die aktuelle Position wird entweder an Gewerk 2 übermittelt (Vektorregelung) oder mit der Sollposition (Positionregelung) verglichen. Der Beobachter wird von Gewerk 3 implementiert und getunt.

2.1 Blockschaltbild

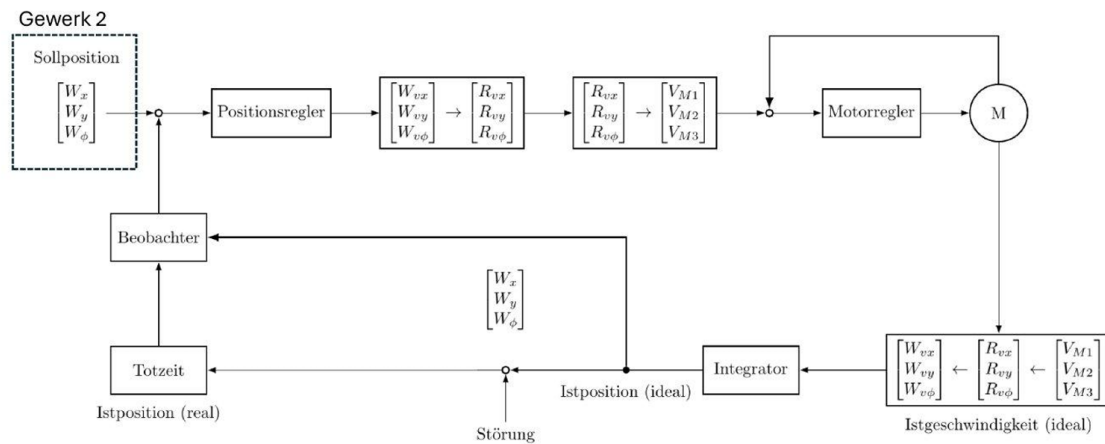


Abbildung 2.1: Blockschaltbild der Positionregelung

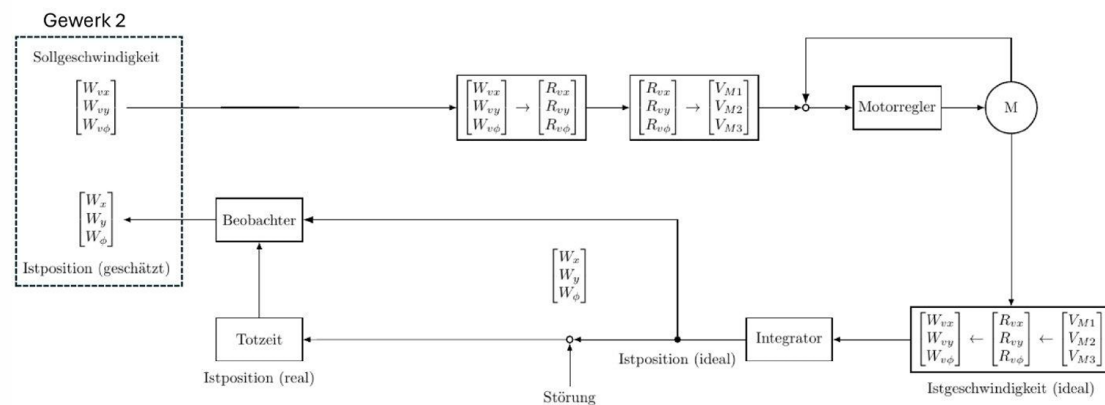


Abbildung 2.2: Blockschaltbild der Geschwindigkeitsregelung

Hier dann auch die Aufteilung in Unterblöcke besprechen

2.2 Schnittstellen

Die Kommunikation zwischen der Bahnplanung (Gewerk 2) und der Robotersteuerung (Gewerk 3) erfolgt über zwei definierte Datenschnittstellen. Über die Schnittstelle von Gewerk 2 zu Gewerk 3 werden Bewegungsaufträge in Form von Fahrvektoren oder Zielpositionen sowie Steuerbefehle zur Ausführung übertragen. Die Robotersteuerung wertet diese Informationen aus und setzt die entsprechenden Bewegungen um.

Datentyp	Name	Kommentar
struct	sFahrvektor	Fahrvektor
Real	reWinkel	Fahrtrichtung
Int	inGeschwindigkeit	Fahrgeschwindigkeit
Int	inControl	0=idle, 1=Stop, 2=Reset, 3=Vektor, 4=Position
struct	sPositions	
Int	inX	X-Position
Int	inY	Y-Position
Real	inWinkel	Ausrichtung

Tabelle 2.1: Gewerk 2 → Gewerk 3

In Gegenrichtung stellt die Robotersteuerung Informationen über die aktuell berechnete Roboterposition sowie den aktuellen Betriebszustand bereit. Dadurch erhält die Bahnplanung kontinuierlich Rückmeldungen über die Position des Roboters und den Fortschritt der ausgeführten Fahrbefehle. Die Trennung von Befehls- und Statusdaten ermöglicht eine klare Strukturierung der Kommunikation zwischen beiden Gewerken und vereinfacht die Erweiterung sowie Wartung des Gesamtsystems.

Datentyp	Name	Kommentar
struct	sKoordinaten	Berechnete Position in Weltkoordinaten
Real	reX	X-Position
Real	reY	Y-Position
Real	reWinkel	Berechnete Ausrichtung des Roboters in Grad
Int	inStatus	0=offline, 1=Betriebsbereit, 2=Laufend, 3=Ziel erreicht, 4=Fehler

Tabelle 2.2: Gewerk 3 → Gewerk 2

2.3 Interne Statemaschine der Regelung

Die Robotersteuerung basiert auf einer zustandsorientierten Ablaufstruktur. Dabei werden die angeforderten Befehle über die Variable **RobotControl** vorgegeben, während der aktuelle Betriebszustand über die Variable **RobotStatus** zurückgemeldet wird.

Variable	Zustände
RobotControl	Vector, Position, Idle, Stop, Reset
RobotStatus	Betriebsbereit, Fehler, Offline, Laufend, Ziel erreicht

Tabelle 2.3: Zustände der internen Statemaschine

Die Variable **RobotControl** dient zur Vorgabe der gewünschten Betriebsart beziehungsweise Bewegung des Roboters. Im Zustand **Idle** befindet sich das System in einem Wartezustand ohne aktive Bewegung. Der Zustand **Position** wird verwendet, um den Roboter auf eine vorgegebene Zielposition zu verfahren. Hierbei wechselt die Regelung auch auf das in Abbildung 2.1 gezeigte Modell. Für Bewegungen entlang eines definierten Bewegungsvektors steht der Zustand **Vector** zur Verfügung, sowie er in Abbildung 2.2 gezeigt wird. Über den Zustand **Stop** können laufende Bewegungen kontrolliert angehalten werden. Der Zustand **Reset** dient dazu, den Roboter in den Grundzustand zurückzusetzen und Fehler zu quittieren.

Die Variable **RobotStatus** beschreibt den aktuellen Betriebszustand des Systems. Der Status **Betriebsbereit** signalisiert, dass keine Störungen vorliegen und neue Bewegungsbefehle verarbeitet werden können. Während der Ausführung eines Fahrbefehls, wenn also der Roboter sich im Zustand **Position** oder **Vector** befindet, wird der Status **Laufend** gesetzt. Nach erfolgreichem Abschluss einer Bewegung wird der Status **Ziel erreicht** ausgegeben. Tritt eine Störung auf, wechselt das System in den Status **Fehler**. Ist keine betriebsbereite Verbindung zum Roboter vorhanden oder steht das System nicht zur Verfügung, wird der Status **Offline** gemeldet.

Durch die Trennung von Steuerbefehlen und Zustandsrückmeldungen wird eine übersichtliche Kommunikation zwischen Anwendungssoftware und Robotersteuerung ermöglicht. Gleichzeitig erlaubt die Struktur eine einfache Erweiterung um zusätzliche Zustände oder Funktionen.

3 Beobachter

Wie in dem Blockschaltbild 2.1 zu sehen wird am Ende nicht die aktuelle Position des Roboters sondern eine verzögerte Position gemessen. Diese Verzögerung entsteht durch die Verarbeitung der Kamera-Messwerte sowie der Versendung über MQTT. Zudem kommt ein neuer Messwert immer nur alle 100 ms (Noch prüfen, ob wir woanders eine Erklärung/Messung dafür anbringen) an, was zu einem Treppenförmigen Signal führt.

3.1 Zielsetzung

Zur Behebung dieser zwei Probleme soll ein Beobachter verwendet werden. Dieser nimmt als Eingang die Raddaten des Roboters und soll uns dann zu jeder Zeit aus der Anfangsposition und den Umdrehungen der Räder, die Aktualposition des Roboters vorwärtspropagieren.

Um dies zu ermöglichen müssen zwei Dinge passieren. Zuerst einmal werden immer die Raddaten der letzten 100 ms benötigt, um die Verzögerung der Kamera auszugleichen. Das heißt bei Eintreffen eines neuen Messwerts muss die erhaltene Position um diese Zeitspanne vorwärtsberechnet werden. Parallel läuft dauerhaft das kinematische Modell mit, welches aus der Anfangsposition und den Raddaten kontinuierlich eine geschätzte Position berechnet. Diese beiden Werte können dann miteinander verglichen werden, um den Fehler zu berechnen und mit einem Gain den Beobachterkreis zu schließen. Dadurch nähert sich schlussendlich der Beobachter und seine Zustandsgrößen dem realen Roboter an.

3.2 Latenzmessung

Die Positionsregelung des omnidirektionalen Roboters wird vollständig auf der internen Steuerung realisiert. Die Rückführung der Roboterposition erfolgt über ein externes Kamerasystem. Aufgrund der unterschiedlichen Abstraten sowie zusätzlicher Verzögerungen durch Bildaufnahme, Bildverarbeitung und Datenübertragung entsteht zwischen der tatsächlichen Bewegung des Roboters und der bereitgestellten Positionsinformation ein zeitlicher Versatz.

Für die Bewertung und Parametrierung der Regelung ist die Kenntnis dieser Systemlatenz von entscheidender Bedeutung. Da die Größe dieser Verzögerung im Vorfeld nicht bekannt war, musste sie experimentell bestimmt werden.

Hierzu wurde der Roboter entlang einer einzelnen Koordinatenachse im eigenen Koordinatensystem mit einer sinusförmigen Bewegung angeregt. Die resultieren-

de Bewegung wurde einerseits durch das Kamerasystem erfasst und andererseits aus den gemessenen Motordrehzahlen beziehungsweise den Inkrementalgebern der Antriebe bestimmt. Durch den Vergleich beider Signalverläufe lässt sich der zeitliche Versatz zwischen der onboard berechneten Roboterbewegung und der externen Positionserfassung bestimmen.

Als Anregungssignal wurde eine Sinusbewegung gewählt, da periodische Signale eine präzise Latenzbestimmung ermöglichen. Insbesondere besitzen sie eindeutig definierte Nullstellen, deren zeitliche Lage bereits bei geringen Verzögerungen messbar verschoben wird. So wird die Systemlatenz durch den zeitlichen Vergleich der Nullstellen beider Geschwindigkeitssignale bestimmt.

Zur Erfassung der Messdaten wurde die Software Beckhoff TwinCAT Scope View eingesetzt. Die Konfiguration der Messaufzeichnung erfolgte mithilfe des Scope Wizards, über den die relevanten Prozesssignale ausgewählt und die Aufzeichnungsparameter festgelegt wurden. Die Kommunikation mit der Steuerung erfolgte über einen bereitgestellten offenen Port, wodurch die Signale der Main Motor Task während des Anlagenbetriebs aufgezeichnet werden konnten.

Während der Messungen wurden die für die Auswertung erforderlichen Größen (die Drehzahlen aller drei Antriebsmotoren sowie die vom Kamerasystem erfassten Positionen des Roboters in X-, Y- und Rotationsrichtung), synchron erfasst.

Nach Abschluss der Messungen wurden die aufgezeichneten Datensätze aus TwinCAT Scope View in das CSV-Format exportiert.

Anschließend erfolgte der Import der Messdaten in MATLAB, wo die weitere Datenvorverarbeitung, Signalverarbeitung und Auswertung durchgeführt wurde. Die MATLAB-Skripte dienten dabei sowohl der Berechnung der Roboterbewegung aus den Motordrehzahlen als auch der Bestimmung der Systemlatenz durch den Vergleich mit den vom Kamerasystem aufgezeichneten Messdaten.

Messwertausfälle des Kamerasystems werden durch lineare Interpolation ersetzt, um kontinuierliche Signalverläufe zu erhalten. Da die Ableitung der Positionsdaten vorhandenes Messrauschen verstärkt, werden sämtliche Signale zusätzlich mit einem Butterworth-Tiefpassfilter zweiter Ordnung mit einer Grenzfrequenz von 2 Hz gefiltert.

Anschließend werden aus den Motordrehzahlen mithilfe der Vorwärtskinematik die translatorischen Geschwindigkeiten in X- und Y-Richtung sowie die Winkelgeschwindigkeit des Roboters berechnet. Die zugrunde liegenden Zusammenhänge sind in Gleichung 3.1 dargestellt.

$$\begin{aligned} v_x &= -\frac{\sqrt{3}}{3}\omega_2 + \frac{\sqrt{3}}{3}\omega_3 \\ v_y &= \frac{2}{3}\omega_1 - \frac{1}{3}\omega_2 - \frac{1}{3}\omega_3 \\ \omega &= \frac{1}{3R}(\omega_1 + \omega_2 + \omega_3) \end{aligned} \tag{3.1}$$

Dabei gilt:

- v_x : translatorische Geschwindigkeit in X-Richtung
- v_y : translatorische Geschwindigkeit in Y-Richtung
- ω : Winkelgeschwindigkeit des Roboters
- $\omega_1, \omega_2, \omega_3$: Drehgeschwindigkeiten der drei Räder
- R : Abstand der Räder zum Roboterzentrum

Je nach Einheit der aufgezeichneten Motordrehzahlen erfolgt anschließend die Umrechnung in konsistente SI-Einheiten. Die Geschwindigkeiten des Kamerasystems werden durch numerische Differentiation der aufgezeichneten Positionsdaten bestimmt. Hierzu wird die MATLAB-Funktion `gradient()` verwendet.

$$\begin{aligned} v_x &= \frac{dx}{dt} \\ v_y &= \frac{dy}{dt} \\ \omega &= \frac{d\theta}{dt} \end{aligned} \quad (3.2)$$

Zur Überprüfung der Berechnung und des Einflusses der Tiefpassfilterung werden die aus den Motordrehzahlen bestimmten Geschwindigkeiten mit den aus den Kamerapositionen berechneten Geschwindigkeiten verglichen. Abbildung 3.1 zeigt die gefilterten und ungefilterten Signalverläufe.

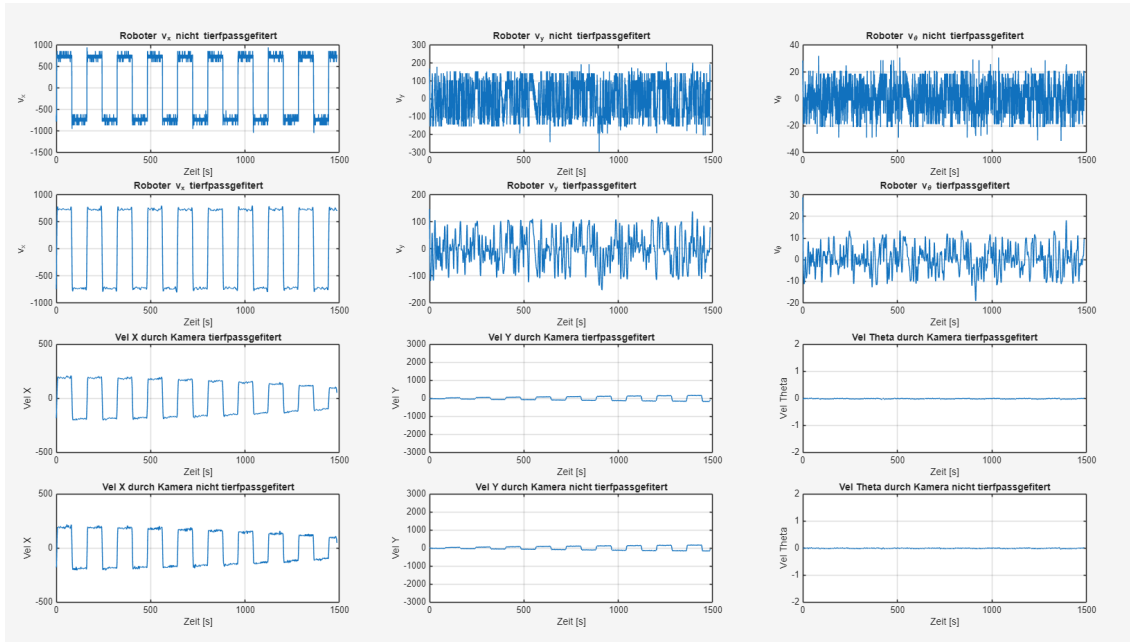


Abbildung 3.1: Vergleich der gefilterten Geschwindigkeitsverläufe aus Motordrehzahlen und Kamerapositionen.

Nach Anwendung der Tiefpassfilterung stimmen beide Signalverläufe deutlich besser überein. Insbesondere die aus den Motordaten berechneten Geschwindigkeiten weisen einen wesentlich ruhigeren Verlauf auf und bilden damit eine geeignete Grundlage für die anschließende Latenzbestimmung.

Bereits aus den Positionsverläufen ist außerdem zu erkennen, dass der Roboter bei einer offenen Ansteuerung ohne Positionsregelung eine kontinuierliche Drift aufweist. Ursache hierfür sind unter anderem Reibungseinflüsse sowie unsymmetrische Abnutzung der Räder. Dadurch entsteht während der Bewegung eine langsame Rotation des Roboters, wodurch zusätzlich ein Bewegungsanteil in Y-Richtung entsteht. In Abbildung 3.2 ist dieser Effekt nochmal besser an dem Verlauf der Position in X und Y-Richtung zu erkennen.

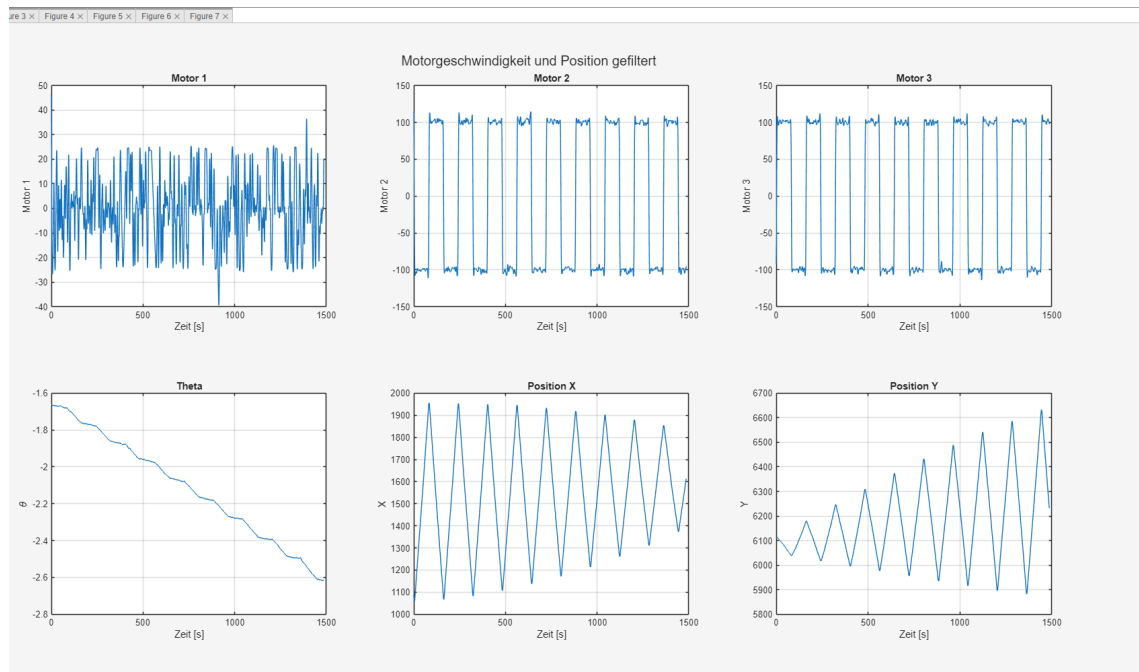


Abbildung 3.2: Positionsverläufe des Roboters bei offener Ansteuerung ohne Positionsregelung.

Zur Bestimmung der Systemlatenz werden die Nullstellen der aus den Motordrehzahlen berechneten sowie der aus den Kamerapositionen bestimmten Geschwindigkeitssignale ausgewertet. Da sich die tatsächlichen Nullstellen in der Regel zwischen zwei diskreten Messpunkten befinden, wird ihre exakte Lage mithilfe einer linearen Interpolation bestimmt. Die Zeitdifferenz entsprechender Nullstellen entspricht unmittelbar der Systemlatenz.

Durch die sinusförmige Testbewegung entstehen zahlreiche Nullstellen, wodurch mehrere unabhängige Latenzmessungen durchgeführt werden können. Aus diesen Einzelwerten werden der Mittelwert und die Standardabweichung berechnet. Der Mittelwert beschreibt die mittlere Systemlatenz, während die Standardabweichung die zeitliche Streuung der Messwerte angibt. Eine geringe Standardabweichung

weist auf eine reproduzierbare und zeitlich weitgehend konstante Verzögerung hin. Geringe Schwankungen entstehen unter anderem durch die endliche Bildrate des Kamerasystems, Unterschiede in der Bildverarbeitung sowie numerische Einflüsse bei der Signalverarbeitung. Der Nullstellenvergleich ist in Abbildung 3.3 nochmals Veranschaulicht. Es ist auch ersichtlich, dass der Roboter seine X-Position in seinen eigenen Koordinaten als Konstant nimmt, während diese in den durch die Kamera aufgezeichneten Weltkoordinaten driftet. Da nur die Nullstellen von Relevanz sind, ist dies relevant.

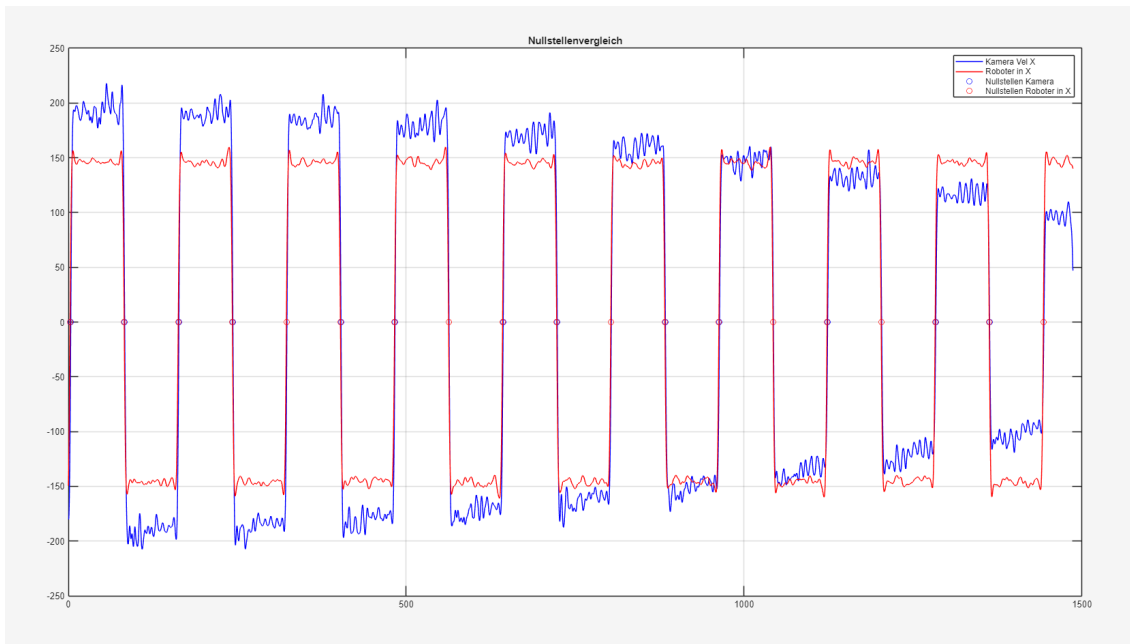


Abbildung 3.3: Vergleich der ermittelten Nullstellen bei der Geschwindigkeitssignale zur Bestimmung der Systemlatenz.

Zur Bestimmung der Systemlatenz wurden mehrere Messungen unter identischen Versuchsbedingungen durchgeführt und mithilfe der entwickelten MATLAB-Auswertung analysiert. Die Ergebnisse der einzelnen Messungen stimmen gut überein und zeigen, dass die Verzögerung zwischen der Roboterbewegung und der kamerabasierten Positionserfassung weitgehend konstant ist.

Aus den ermittelten Einzelmessungen ergibt sich eine mittlere Systemlatenz von 100 ms. Die geringe Streuung bestätigt die Reproduzierbarkeit der entwickelten Messmethode und zeigt, dass die Latenz im betrachteten Messbereich als nahezu konstanter Zeitversatz angenommen werden kann. Dieser Zeitversatz kann bei der weiteren Parametrierung und Bewertung der Regelung gezielt berücksichtigt werden.

3.3 Kinematisches Beobachtermodell

Der Roboter verfügt über drei omnidirektionale Räder, die in einem Winkel von $\alpha_1 = 60^\circ$, $\alpha_2 = 180^\circ$ und $\alpha_3 = 300^\circ$ angeordnet sind. Jedes Rad erzeugt eine Geschwindigkeit entlang seiner Rollrichtung sowie eine Querkomponente, die durch die Kopplung mit den beiden anderen Rädern entsteht.

Wie in Abbildung 3.4 zu sehen, sind die Geschwindigkeitsvektoren der Räder tangential zum Kreis des Roboters angeordnet und entsprechen den jeweiligen Radgeschwindigkeiten V_{w1} , V_{w2} und V_{w3} .

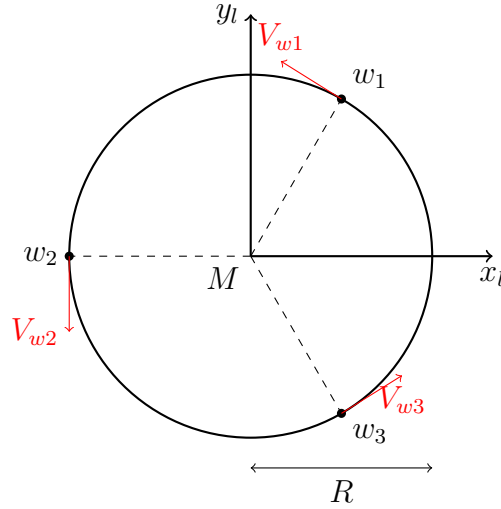


Abbildung 3.4: 3-Rad-Omniroboter mit Radwinkeln 60° , 180° , 300°

Um die Gesamtgeschwindigkeit des Roboters zu berechnen, müssen die Beiträge aller drei Räder berücksichtigt und in das globale Koordinatensystem transformiert werden. Hierfür wird zunächst die Geschwindigkeit jedes Rades im lokalen Radkoordinatensystem beschrieben, wobei die erste Zeile die Querkomponente und die zweite Zeile die direkte Rollkomponente darstellt. Die Querkomponente entsteht durch die Kopplung mit den anderen Rädern und lässt sich berechnen, indem die Geschwindigkeitsvektoren der anderen Räder entsprechend ihrer Anordnung und der Geometrie des Roboters gewichtet werden. Da die Räder in einem Winkel von 120° zueinander angeordnet sind, ergibt sich für die Querkomponente ein Faktor von $1/\sqrt{3}$, wie in [2] hergeleitet :

$$\vec{V}_1^{wheel} = \begin{bmatrix} \frac{V_{w3}}{\sqrt{3}} - \frac{V_{w2}}{\sqrt{3}} \\ V_{w1} \end{bmatrix} \quad (3.3)$$

$$\vec{V}_2^{wheel} = \begin{bmatrix} \frac{V_{w1}}{\sqrt{3}} - \frac{V_{w3}}{\sqrt{3}} \\ V_{w2} \end{bmatrix} \quad (3.4)$$

$$\vec{V}_3^{wheel} = \begin{bmatrix} \frac{V_{w2}}{\sqrt{3}} - \frac{V_{w1}}{\sqrt{3}} \\ V_{w3} \end{bmatrix} \quad (3.5)$$

Die zweite Zeile beschreibt die Geschwindigkeit entlang der Rollrichtung des jeweiligen Rades, während die erste Zeile die Querkomponente darstellt, die durch die Kopplung mit den anderen Rädern entsteht. Diese lokalen Geschwindigkeiten werden dann, über die Rotationsmatrix, in das Roboterkoordinatensystem transformiert:

$$\vec{V}_j = \mathbf{R}(\alpha_j) \vec{V}_j^{(w)} \quad (3.6)$$

mit der Rotationsmatrix:

$$\mathbf{R}(\alpha_j) = \begin{bmatrix} \cos \alpha_j & -\sin \alpha_j \\ \sin \alpha_j & \cos \alpha_j \end{bmatrix} \quad (3.7)$$

Die Gesamtgeschwindigkeit des Roboters ergibt sich als Mittelwert aller drei Räder:

$$\begin{bmatrix} v_x^{body} \\ v_y^{body} \end{bmatrix} = \frac{1}{3} \sum_{j=1}^3 \mathbf{R}(\alpha_j) \vec{V}_j^{(w)} \quad (3.8)$$

Und die Rotationsgeschwindigkeit ist:

$$\dot{\varphi} = \frac{1}{3R} (V_{w1} + V_{w2} + V_{w3}) \quad (3.9)$$

Um diese allgemeinen Gleichungen auf den konkreten Roboter anzuwenden, müssen die Winkelwerte $\alpha_1 = 60^\circ$, $\alpha_2 = 180^\circ$ und $\alpha_3 = 300^\circ$ eingesetzt werden.

Durch Einsetzen dieser Werte in die Rotationsmatrizen und Ausmultiplizieren der Summe ergibt sich die folgende kompakte Matrixdarstellung:

$$\begin{bmatrix} v_x^{body} \\ v_y^{body} \\ \dot{\varphi} \end{bmatrix} = \begin{bmatrix} -\frac{\sqrt{3}}{3} & 0 & \frac{\sqrt{3}}{3} \\ \frac{1}{3} & -\frac{2}{3} & \frac{1}{3} \\ \frac{1}{3R} & \frac{1}{3R} & \frac{1}{3R} \end{bmatrix} \begin{bmatrix} V_{w1} \\ V_{w2} \\ V_{w3} \end{bmatrix} \quad (3.10)$$

Die Umrechnung Geschwindigkeit in Weltkoordinaten erfolgt dabei wie folgt, wobei θ der aktuelle Winkel des Roboters im Raum ist:

$$\begin{bmatrix} v_x^{world} \\ v_y^{world} \end{bmatrix} = \begin{bmatrix} \cos(\theta) & -\sin(\theta) \\ \sin(\theta) & \cos(\theta) \end{bmatrix} \begin{bmatrix} v_x^{body} \\ v_y^{body} \end{bmatrix} \quad (3.11)$$

Die Winkelgeschwindigkeit des Roboters ist in Roboterkoordinaten, als auch Weltkoordinaten gleich definiert, weshalb hier keine Unterscheidung vorgenommen wird.

Um schlussendlich die Position im Raum zu berechnen, integriert man die Geschwindigkeiten über die Zeit. Bei einer diskreten Integration mit einem Zeitschritt Δt ergibt sich die folgende Formel für die Aktualisierung der Position:

$$\begin{bmatrix} p_{x+1}^{world} \\ p_{y+1}^{world} \\ p_{\varphi+1} \end{bmatrix} = \begin{bmatrix} p_x^{world} \\ p_y^{world} \\ p_{\varphi} \end{bmatrix} + \Delta t \cdot \begin{bmatrix} v_x^{world} \\ v_y^{world} \\ \omega \end{bmatrix} \quad (3.12)$$

3.4 Beobachterstruktur

Hier vielleicht noch kurz iwas angeben zu, warum Struktur oder so

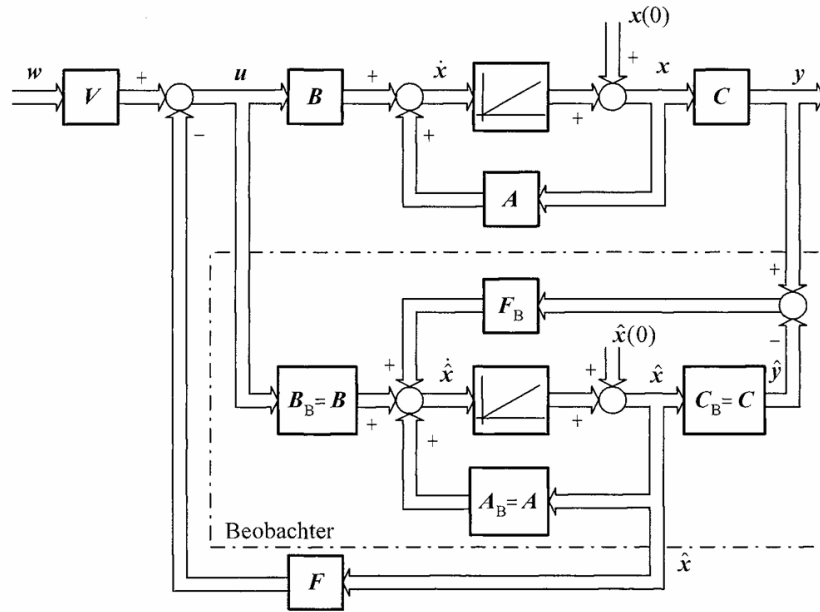


Abbildung 3.5: Zustandsbeobachter nach Luenberger [3]

Die erste Abbildung zeigt die klassische Struktur eines Zustandsbeobachters nach Luenberger. Ausgangspunkt ist ein lineares zeitinvariantes System in Zustandsraumdarstellung mit den Matrizen A , B und C . Ziel des Beobachters ist die Rekonstruktion der nicht direkt messbaren Zustände x auf Basis des Eingangssignals u und der gemessenen Ausgangsgröße y . Der Luenberger-Beobachter bildet dazu ein dynamisches System nach, das strukturell weitgehend identisch mit dem Originalsystem ist. Der zentrale Unterschied besteht in der Rückführung des Schätzfehlers, also der Differenz zwischen gemessenem Ausgang y und geschätztem Ausgang $\hat{y}$. Diese Differenz wird über eine Beobacherverstärkung F (im Folgenden L genannt) auf das Beobachtersystem zurückgeführt. Dadurch ergibt sich die typische Beobachterdynamik: $\dot{\hat{x}} = A\hat{x} + Bu + F(y - \hat{y})$ mit $\hat{y} = C \cdot \hat{x}$. Die Aufgabe bei der Auslegung des Luenberger-Beobachters besteht darin, die Verstärkungsmatrix L so zu wählen, dass

der Schätzfehler $e = x - \hat{x}$ asymptotisch gegen null konvergiert. Anschaulich bedeutet dies, dass sich der Zustandsbeobachter an unser reales Modell möglichst gut annähert.

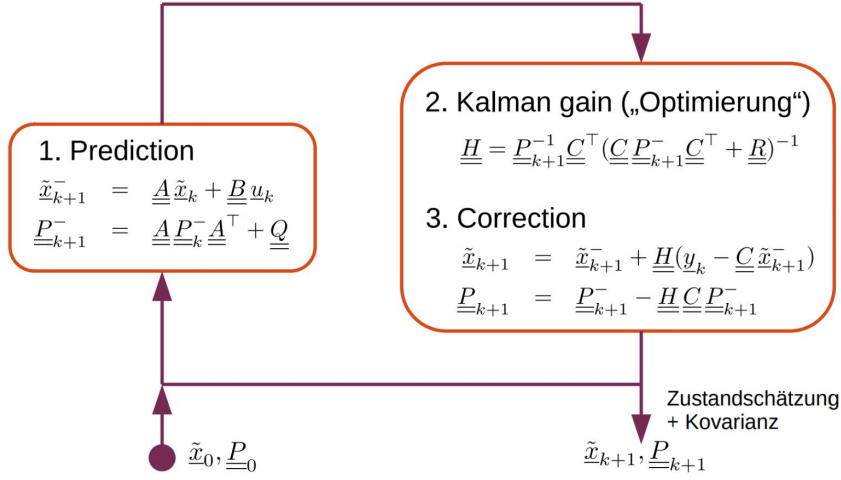


Abbildung 3.6: Konzept Kalman-Filter [1]

Die Abbildung 3.6 zeigt das vereinfachte Konzept des Kalman-Filters, das im Folgenden als Erweiterung des Luenberger-Beobachters untersucht werden soll. Auch hier erfolgt eine Zustandsrekonstruktion, jedoch auf Basis einer stochastischen Beschreibung von Prozess- und Messrauschen. Der Kalman-Filter erweitert die Beobachterstruktur um eine stochastische Beschreibung von Prozess- und Messrauschen. Die Zustandsrekonstruktion erfolgt in zwei Schritten. Dabei wird in der Prädiktion auf Basis des Systemmodells der Zustand sowie die Fehlerkovarianz vorhergesagt:

$$\hat{x}_{k+1}^- = A\hat{x}_k + Bu_k \quad (3.13)$$

$$P_{k+1}^- = AP_kA^T + Q \quad (3.14)$$

In der Korrektur wird dann zyclisch die Vorhersage mithilfe der Messung y_{k+1} korrigiert. Der Kalman-Gewinn ergibt sich dabei zu:

$$K_{k+1} = P_{k+1}^- C^T (C P_{k+1}^- C^T + R)^{-1} \quad (3.15)$$

Damit werden Zustandsschätzung und Kovarianz aktualisiert:

$$\hat{x}_{k+1} = \hat{x}_{k+1}^- + K_{k+1} (y_{k+1} - C\hat{x}_{k+1}^-) \quad (3.16)$$

$$P_{k+1} = (I - K_{k+1}C) P_{k+1}^- \quad (3.17)$$

Der Kalman-Filter bestimmt den Verstärkungsfaktor K_{k+1} optimal im Sinne der Minimierung der Schätzfehlerkovarianz. Dabei werden die statistischen Eigenschaften des Prozessrauschens (Q) und des Messrauschens (R) explizit berücksichtigt. Der wesentliche Unterschied zwischen dem Luenberger-Beobachter und dem Kalman-Filter liegt in der Auslegung der Beobachterverstärkung. Beim Luenberger-Beobachter

wird die Matrix L deterministisch gewählt, typischerweise über Polvorgabe. Rauscheinflüsse werden nicht explizit berücksichtigt. Beim Kalman-Filter ergibt sich der Verstärkungsfaktor als Lösung eines Optimierungsproblems und wird in jedem Zeitschritt neu berechnet. Dabei werden Unsicherheiten systematisch über Kovarianzmatrizen modelliert.

Im Rahmen des Verbundprojekts wurden die Reglerstruktur sowie die Berechnungsgrundlagen so ausgelegt, dass eine spätere Erweiterung auf eine zur Laufzeit veränderliche Beobacherverstärkung ohne größeren Aufwand möglich wäre. Darüber hinaus wurde der Algorithmus des Kalman-Filters bereits implementiert und für den Einsatz vorbereitet. Im Zuge der Untersuchungen zeigte sich jedoch, dass das Messrauschen der Kamera sehr gering ist und der Messfehler nur minimale Schwankungen aufweist. Da ein Kalman-Filter seine Vorteile insbesondere bei verrauschten Messsignalen ausspielt, ist unter diesen Randbedingungen lediglich ein geringer zusätzlicher Nutzen zu erwarten. Zusätzlich ist die Aktualisierungsrate der Kamera im Vergleich zur Zykluszeit der Regelungssoftware von 0,3333 ms sehr niedrig. Somit stehen nur vergleichsweise selten neue Messwerte zur Verfügung, wodurch das Potenzial einer optimierten Zustandsschätzung weiter eingeschränkt wird. Darüber hinaus erfordert die Parametrierung eines Kalman-Filters eine sorgfältige Abstimmung der Prozess- und Messrauschkovarianzen. Da unter den gegebenen Hardwarebedingungen kein nennenswerter Genauigkeitsgewinn zu erwarten war, wurde auf eine weitere Optimierung des Kalman-Filters verzichtet. Stattdessen wurde eine deterministische Auslegung der Beobacherverstärkung gewählt, die ein vorhersehbares Verhalten ermöglicht und die Anforderungen der Anwendung bereits vollständig erfüllt.

3.5 Modell Validierung

Um das Beobachter Modell und dessen Implementierung zu validieren, ist es am einfachsten zuerst die Normalfahrt zu betrachten. Also eine Fahrt in der zyklisch neue Kamera-Werte ankommen. Der Verlauf dieser Messwerte ist in Abbildung 3.7, beispielhaft für die x-Position des Roboters, in Blau dargestellt.

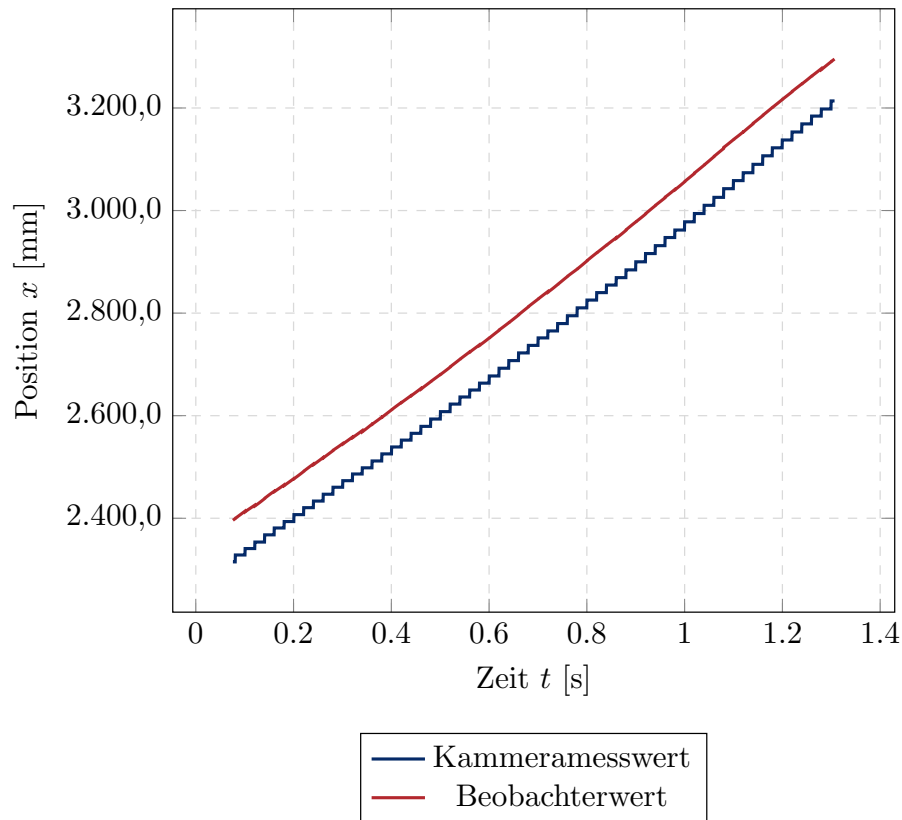


Abbildung 3.7: Beobachter mit direktem Ausgleich Normalfahrt

Betrachtet man im Gegensatz zur blauen Kurve die roten, vom Beobachter berechneten Werte und vergleicht diese mit den in der Zielsetzung beschriebenen Anforderungen, lässt sich ein nahezu ideales Verhalten erkennen. Die Position wird um 100ms vorwärts propagiert, und zwischen den Sprüngen wird die Kurve geglättet.

Noch deutlicher wird dies, wenn man die Beobachterkurve künstlich um 100 ms verzögert. Dies ist in Abbildung 3.8 dargestellt. Hier ist gut zu erkennen, wie die Kamera bei jedem neuen Messwert wieder exakt auf die kontinuierliche Kurve des Beobachters springt.

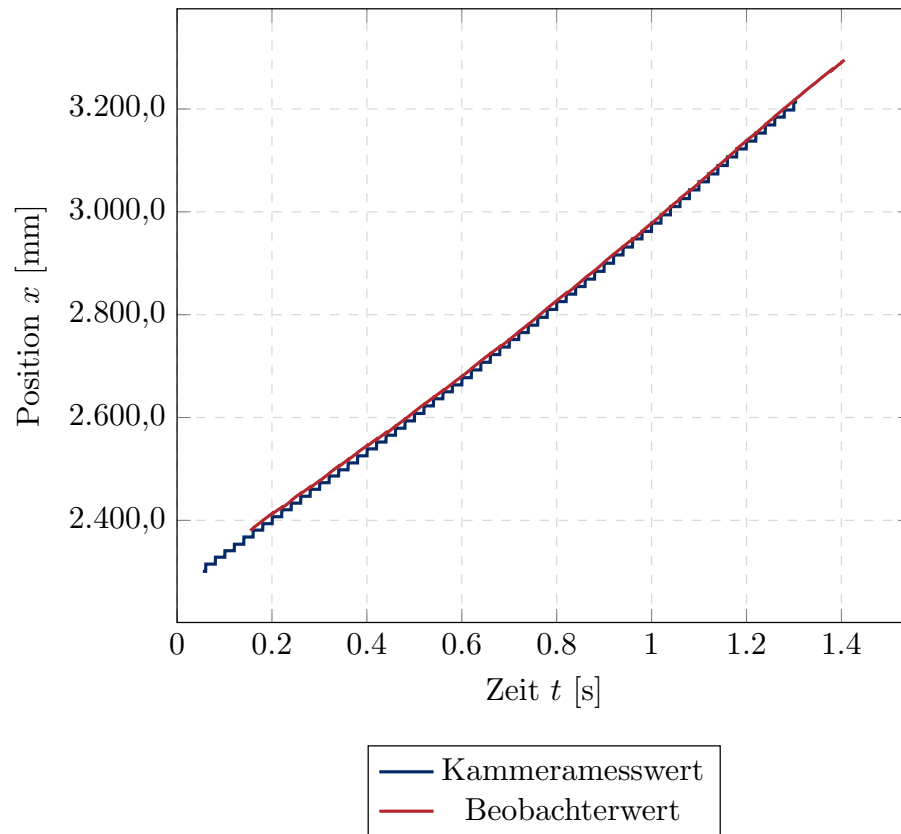


Abbildung 3.8: Beobachter mit direktem Ausgleich Normalfahrt Vershoben

3.6 Beobacherverstärkung Paremetrierung

Für die Regler Parametrierung soll nun eine Fahrt betrachtet werden, in der der Roboter eine längere Zeit keinen Kammera-Messwert bekommt und somit das Modell langsam von der Realität abdriftet. In Abbildung 3.9 ist diese Messung einmal im gesamten dargestellt. Hierfür wurde während der Fahrt in x-Richtung der Navigationscode des Roboters für etwa 12s abgedeckt.

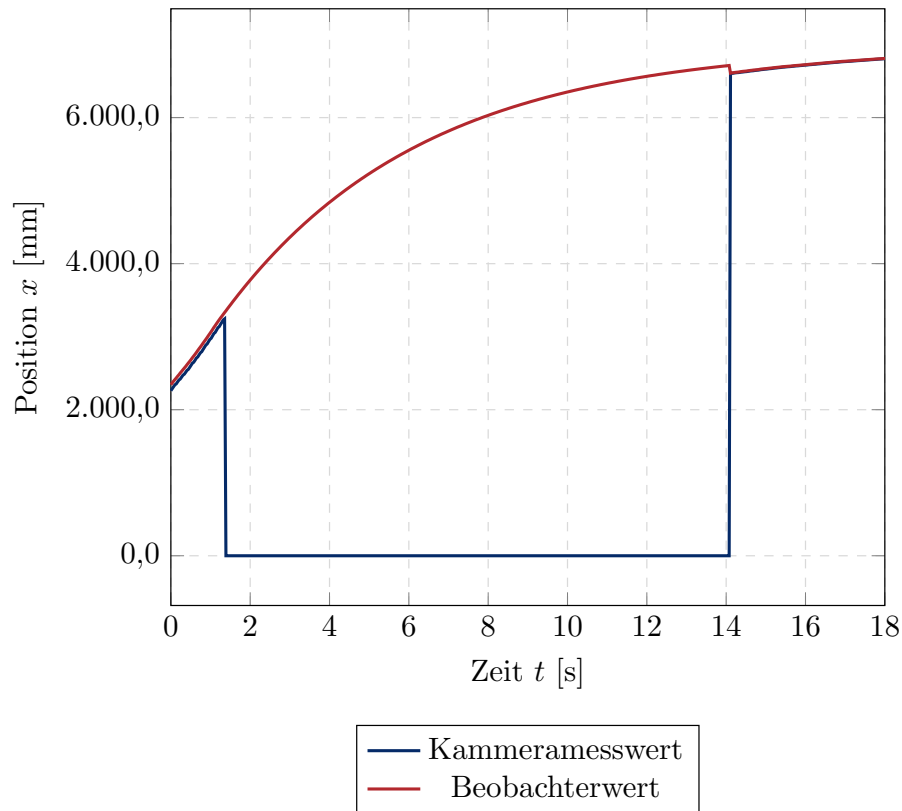


Abbildung 3.9: Beobachter mit direktem Ausgleich Gesamt

In Abbildung 3.10 ist der Zeitbereich um den Ausgleich nochmal vergrößert dargestellt. Wie man hier gut erkennen kann, wurde der Gain der Rückführung hier auf eins gesetzt, was besonders durch die schnellen Zykluszeiten des Programms zu einem quasi direkten Ausgleich bei Erhalt eines neuen Kammeramesswerts führt.

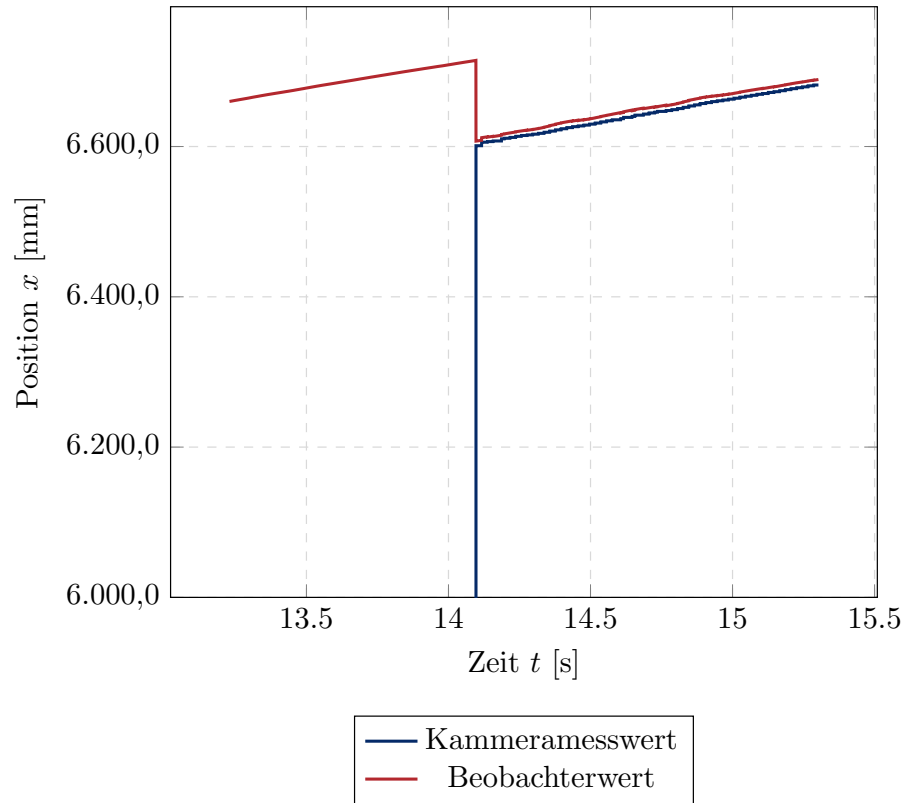


Abbildung 3.10: Beobachter mit direktem Ausgleich

Betrachtet man den gleichen Versuch nochmal für einen deutlich geringere Verstärkung, so kann man in Abbildung 3.11, für $L = 0.01$ eine Annäherung an den Kammerawert erkennen.

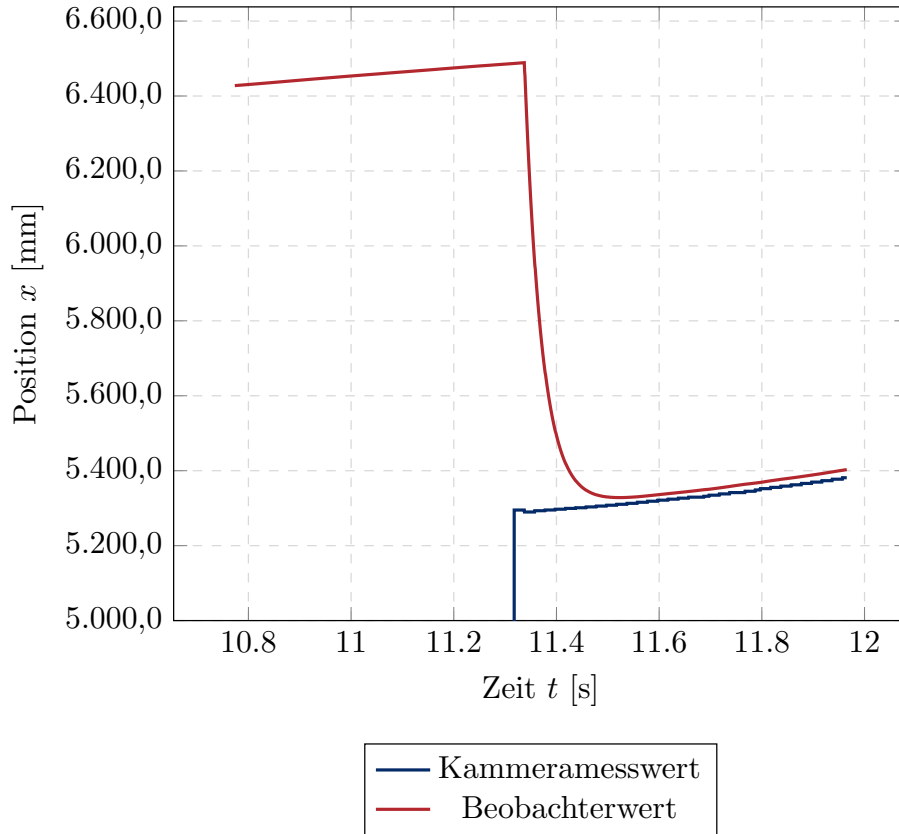


Abbildung 3.11: Beobachter mit langsamen Ausgleich

Zu guter Letzt soll anhand von Abbildung 3.12 untersucht werden, ob durch eine vom Kamerafehler abhängige variable Beobacherverstärkung eine weitere Verbesserung des Beobachterverhaltens erreicht werden kann. Die zugrunde liegende Idee orientiert sich an einem vereinfachten Grundgedanken des Kalman-Filters. Besitzt die Kamera eine hohe Messgüte, soll den Messwerten ein höheres Vertrauen entgegengebracht werden. Nimmt die Qualität der Messung hingegen ab, soll der Beobachter stärker auf das zugrunde liegende Modell zurückgreifen. Da der von der Kamera bereitgestellte Fehlerindikator direkt verfügbar ist, bietet sich dessen Verwendung zur Anpassung der Beobacherverstärkung an. Hierzu wird der Kamerafehler e auf einen geeigneten Wertebereich normiert und zur Skalierung der Verstärkung herangezogen. Zur Umsetzung wird ein Schwellwert $E_{th} = 5$ eingeführt. Dieser Wert entspricht dem maximal beobachteten Kamerafehler, bei dem in der Anlagensimulation noch kein vollständiger Ausfall der Kamera festgestellt wurde. Für Fehlerwerte im Bereich $E_{th}0 < e < E_{th}$ wird die Beobacherverstärkung proportional zum normierten Fehler E_{th} skaliert. Dadurch erfolgt eine kontinuierliche Anpassung der Verstärkung in Abhängigkeit von der Qualität der Kameramessung. Überschreitet der Kamerafehler den Schwellwert E_{th} , wird von einem fehlerhaften beziehungsweise nicht mehr vertrauenswürdigen Kamerasignal ausgegangen. Um eine übermäßige Beeinflussung des Beobachters durch fehlerhafte Messwerte zu vermeiden, wird die

Verstärkung in diesem Fall auf einen empirisch bestimmten Maximalwert begrenzt. Die resultierende Verstärkung ergibt sich damit zu

$$L(e) = \begin{cases} L_{\text{nom}} \frac{e}{E_{th}}, & 0 \leq e \leq E_{th} \\ L_{\text{nom}}, & e > E_{th} \end{cases} \quad (3.18)$$

wobei L_{nom} die nominale beziehungsweise maximal zulässige Beobacherverstärkung beschreibt.

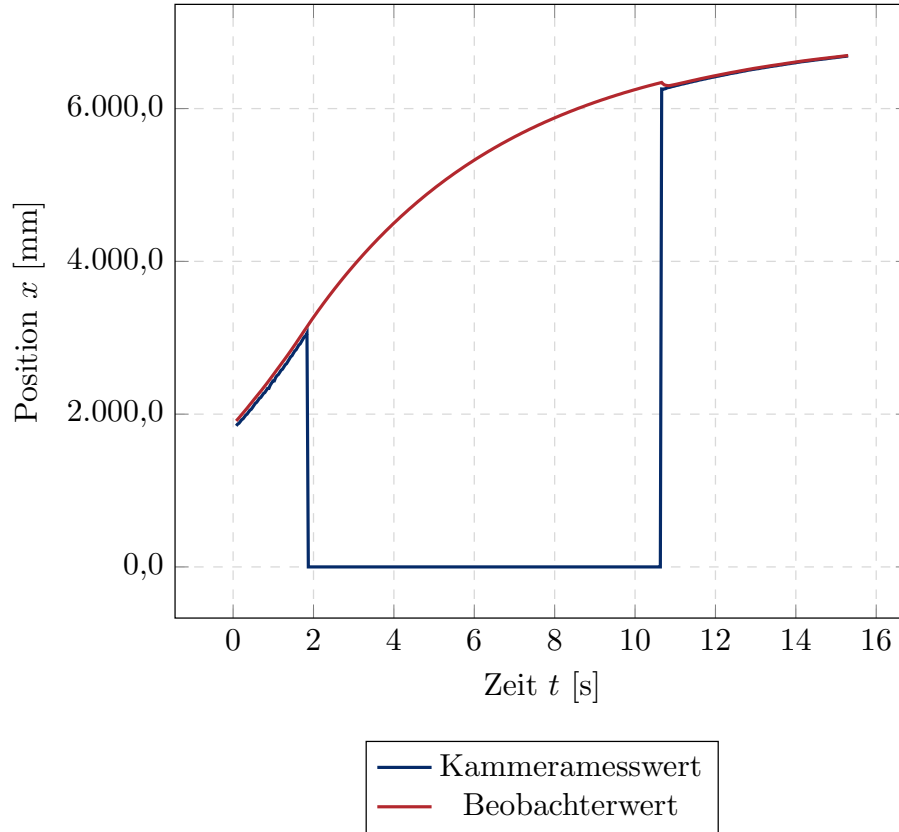


Abbildung 3.12: Beobachter mit variablen Gain

Der Ansatz ist grundsätzlich sinnvoll, zeigt in der Praxis jedoch nur einen geringen Effekt. Dies liegt daran, dass die Kamera bei jeder erneuten Bilderkennung einen sehr ähnlichen systematischen Fehler aufweist. Eine dynamische Korrektur bringt daher nur begrenzte Verbesserungen. Aus diesem Grund erscheint es zielführender, den Fehler durch Anpassung eines statischen Parameters zu kompensieren. Durch ein gezieltes Tuning dieses festen Wertes kann eine vergleichbare oder sogar stabilere Verbesserung erzielt werden, ohne die zusätzliche Komplexität einer dynamischen Anpassung.

4 Regler

Das Kapitel arbeitet sich von der untersten Reglerebene am Rad bis zum übergeordneten Lageregler vor und folgt damit der Signalkette aus Abbildung Konzept in umgekehrter Richtung. Den Anfang macht der Drehzahlregler, der an jedem der drei Räder dafür sorgt, dass die vorgegebene Sollgeschwindigkeit anliegt. Da Gewerk 2 jedoch keine Raddrehzahlen, sondern eine Sollgeschwindigkeit des Roboters liefert, ist eine Transformation von Roboter in Motorkoordinaten nötig. Damit sind alle Bausteine für die Vektorfahrweise beisammen. Für die Positionsfahrweise kommt eine weitere Ebene hinzu: Die gemessene Position liegt in Weltkoordinaten vor und muss in das Roboterkoordinatensystem überführt werden. Darauf setzt schließlich der Lageregler auf, dessen Parameter über ein Gain-Scheduling an die jeweilige Situation angepasst werden, um eine zuverlässige Fahrt sicherzustellen.

4.1 Drehzahlregler

Der Drehzahlregler sorgt dafür, dass eine vorgegebene Sollgeschwindigkeit des Roboters auf die Motorbewegung der einzelnen Motoren übersetzt wird und korrekt umgesetzt wird. Der Drehzahlregler wird in TwinCAT als Funktionsbaustein umgesetzt. Als Eingangsgrößen werden Sollgeschwindigkeiten in X,Y, und θ im Roboterkoordinatensystembenötigt. Außerdem werden die Istgeschwindigkeiten der einzelnen Motoren an den Eingang zurückgeführt. Der Ausgang ist das Geschwindigkeitessignal für die einzelnen drei Motoren. Abbildung 4.1 zeigt den Drehzahlregler als Blackbox mit Eingangs- und Ausgangssignalen.

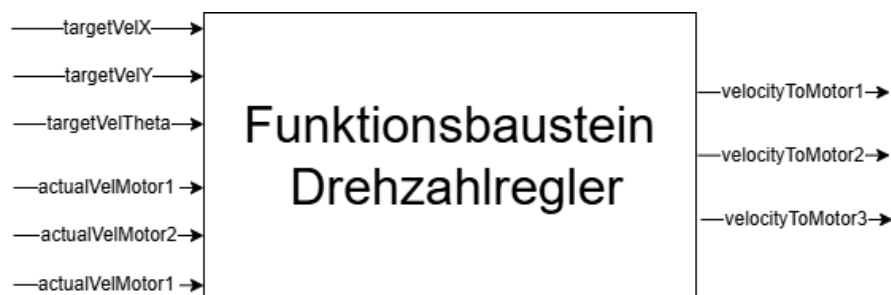


Abbildung 4.1: Drehzahlregler als Blackbox mit Eingangs- und Ausgangssignalen

4.1.1 Ein- und Ausgangssignale

TargetVelocity

Die Target Velocity ist die Sollgeschwindigkeit des Roboters im Koordinatensystem des Roboters. Sie wird in $\frac{m}{s}$ angegeben und besteht aus drei voneinander unabhängigen Komponenten: X, Y und θ . Die X-Komponente beschreibt die Geschwindigkeit in Vorwärtsrichtung, die Y-Komponente die Geschwindigkeit in Querrichtung und die θ -Komponente die Winkelgeschwindigkeit um die Z-Achse. Durch Überlagerung der Bewegungen ist der Roboter in der Lage sich in alle Richtungen im Raum zu bewegen.

VelWheel

VelWheel1, -2 und -3 ist die Istgeschwindigkeit der einzelnen Motoren. Sie wird auch in m/s angegeben. Die Berechnung erfolgt über den Encoder des Motors und den Radumfang mit folgender Formel:

$$\text{VelWheel1} = - \frac{n_{1,\text{fromMotor}}}{n_{\text{max,KlemmeDrehzahlMessung}}} \cdot \frac{n_{\text{max,MotorRPM}}}{60} \cdot \ddot{u} \cdot 2\pi \cdot r_{\text{Wheel}} \quad (4.1)$$

Dabei bezeichnen:

- $n_{1,\text{fromMotor}}$ – Rohwert der Drehzahlmessung an der Motorklemme (vorzeichenbehaftet).
- $n_{\text{max,KlemmeDrehzahlMessung}} = 10\,000$ – Wert, den die Klemme bei maximaler Motordrehzahl zurückliefert (Skalierungsreferenz der Messung).
- $n_{\text{max,MotorRPM}} = 3600 \text{ rpm}$ – maximale Motordrehzahl. Zusammen mit dem vorherigen Term wird der Rohwert auf eine tatsächliche Drehzahl in Umdrehungen pro Minute umgerechnet; die Division durch 60 wandelt diese in Umdrehungen pro Sekunde um.
- $\ddot{u} = 0,0625$ – Getriebeübersetzung zwischen Motor- und Raddrehzahl (Ausgangsdrehzahl = Motordrehzahl $\cdot \ddot{u}$).
- $r_{\text{Wheel}} = 0,040 \text{ m}$ – Radradius. Über $2\pi \cdot r_{\text{Wheel}}$ wird die Raddrehzahl (in Umdrehungen/s) in eine Umfangsgeschwindigkeit umgerechnet.

VelocityToMotor

VelocityToMotor1, -2 und -3 ist das jeweilige Ausgangssignal des Drehzahlreglers an die drei Motoren. Da die Motoren als Sollwert keinen Geschwindigkeitswert in mm/s, sondern einen skalierten Integerwert erwarten, muss VelWheel vor der Ausgabe mit einem Skalierungsfaktor multipliziert werden. Die Berechnung erfolgt mit folgender Formel:

$$\text{VelocityToMotor1} = \text{VelWheel1} \cdot \text{skalPos} \quad (4.2)$$

Dabei bezeichnen:

- VelWheel1 – Istgeschwindigkeit des Motors in mm/s (siehe oben).

- $\text{skalPos} = 34,7668668 \left[\frac{1}{\text{mm/s}} \right]$ – Skalierungsfaktor, der die Geschwindigkeit in mm/s auf den vom Motor erwarteten Integer-Wertebereich abbildet. Er berechnet sich aus dem Verhältnis von maximalem Ausgabewert zu maximal erreichbarer Geschwindigkeit:

$$\text{skalPos} = \frac{\text{MaxValue}}{\text{MaxVelocity}} \quad (4.3)$$

- $\text{MaxValue} = 32767$ – maximaler darstellbarer Ausgabewert (Wertebereich eines vorzeichenbehafteten 16-Bit-Integers).
- $\text{MaxVelocity} = 942 \text{ mm/s}$ – maximal erreichbare Geschwindigkeit des Rades, berechnet aus dem Raddurchmesser D , der maximalen Motordrehzahl n und der Getriebeübersetzung:

$$\text{MaxVelocity} = \frac{D \cdot \pi \cdot n}{16 \cdot 60} \quad (4.4)$$

- D – Raddurchmesser in mm.
- n – maximale Motordrehzahl in rpm.
- 16 – Getriebeübersetzung (Motor- zu Raddrehzahl, entspricht $\ddot{u} = \frac{1}{16} = 0,0625$).
- 60 – Umrechnungsfaktor von Minuten auf Sekunden.

Da VelWheel1 in mm/s und skalPos in $\frac{1}{\text{mm/s}}$ angegeben sind, ist VelocityToMotor1 dimensionslos und stellt den direkten Sollwert für den Motorregler dar.

4.1.2 Umrechnung von Roboter- in Motorkoordinaten

Die von Gewerk 2 gelieferte Sollgeschwindigkeit liegt als kartesischer Geschwindigkeitsvektor im Koordinatensystem des Roboters vor (TargetVelocity , siehe oben) und muss für die einzelnen Motoren in eine jeweilige Solldrehzahl umgerechnet werden. Da der Roboter über drei omnidirektionale Räder verfügt, die jeweils um $\frac{2\pi}{3}$ versetzt am Roboter angebracht sind, ergibt sich die Solldrehzahl jedes Motors aus einer Überlagerung der drei Bewegungskomponenten $\dot{x}$, $\dot{y}$ und $\dot{\theta}$.

Bevor die Transformation durchgeführt wird, werden die Sollgeschwindigkeiten zunächst invertiert:

$$\dot{x} = -\text{targetVelX}, \quad \dot{y} = -\text{targetVelY}, \quad \dot{\theta} = -\text{targetVelTheta} \quad (4.5)$$

Diese Invertierung ist notwendig, da die Drehrichtung der Motoren im Projekt invers zur Definition der Sollgeschwindigkeit festgelegt ist. Ohne diese Anpassung würde der Roboter bei positiv vorgegebener Sollgeschwindigkeit in die jeweils entgegengesetzte

Richtung fahren bzw. sich entgegengesetzt zur vorgegebenen Winkelgeschwindigkeit drehen.

Die eigentliche Umrechnung von Roboter- in Motorkoordinaten erfolgt anschließend über eine kinematische Transformationsmatrix, die ausschließlich aus konstanten Koeffizienten besteht:

$$\begin{pmatrix} \text{targetVelocityMotor1} \\ \text{targetVelocityMotor2} \\ \text{targetVelocityMotor3} \end{pmatrix} = \begin{pmatrix} -\sin\left(\frac{\pi}{3}\right) & \cos\left(\frac{\pi}{3}\right) & R_{\text{Robot}} \\ -\sin(\pi) & \cos(\pi) & R_{\text{Robot}} \\ -\sin\left(\frac{5\pi}{3}\right) & \cos\left(\frac{5\pi}{3}\right) & R_{\text{Robot}} \end{pmatrix} \begin{pmatrix} \dot{x} \\ \dot{y} \\ \dot{\theta} \end{pmatrix} \quad (4.6)$$

Dabei bezeichnen:

- $\frac{\pi}{3}, \pi, \frac{5\pi}{3}$ – Winkelposition der jeweiligen Räder am Roboter, bezogen auf die Vorwärtsrichtung des Roboters. Die Räder sind gleichmäßig um jeweils $\frac{2\pi}{3}$ versetzt angeordnet.
- R_{Robot} – Radius des Roboters, also der Abstand vom Mittelpunkt des Roboters zu den Rädern. Da $\dot{x}$ und $\dot{y}$ in mm/s vorliegen, wird R_{Robot} (intern in Metern definiert, `GVL_RobotConstant.ROBOT_RADIUS`) mit dem Faktor 1000 ebenfalls in Millimeter umgerechnet, um Einheitskonsistenz sicherzustellen. Da R_{Robot} für alle drei Zeilen der Matrix denselben, konstanten Wert annimmt, bildet die dritte Spalte der Matrix ausschließlich diesen einen Koeffizienten ab.
- $\dot{x}, \dot{y}, \dot{\theta}$ – invertierte Sollgeschwindigkeiten des Roboters (siehe oben).

Das Ergebnis dieser Transformation, `targetVelocityMotor1`, -2 und -3, entspricht der jeweiligen Solldrehzahl der drei Motoren und dient als Eingangsgröße für den in Abschnitt 4.1.3 beschriebenen PI-Drehzahlregler. Die exemplarische Umsetzung dieser Umrechnung in TwinCAT wird im folgenden Abschnitt anhand einer Tabelle dargestellt.

4.1.3 Interne Reglerstruktur

Der Drehzahlregler ist als herkömmlicher PI-Regler ausgelegt. Er bildet aus den beiden Eingangssignalen Sollwert und Istwert einen Regelfehler, welcher über die beiden Reglerpfade proportional und integral verarbeitet wird. Der Regler bedient die 3 Motoren unabhängig voneinander, sodass die Reglerstruktur aus 3 parallelen aber identischen SISO Reglern besteht. Abbildung 4.2 zeigt die interne Struktur des Drehzahlreglers für einen Motor.

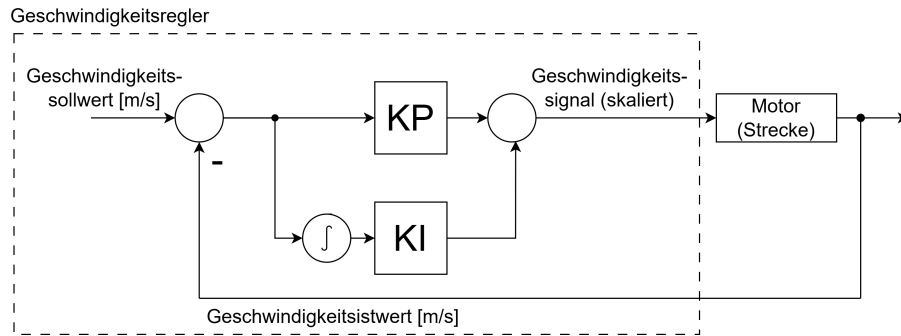


Abbildung 4.2: Interne Struktur des PI-Drehzahlreglers für einen Motor

Der Regler ist als PI-Regler ausgelegt, um eine bleibende Regelabweichung zu eliminieren. Die Regelparameter sind experimentell ermittelt worden. Da die Reglerperformance mit diesen Parametern für alle drei Motoren zufriedenstellend ist, wurde auf eine quantitative Optimierung verzichtet. Die Parameter sind in Tabelle 4.1.3 zusammengefasst. Die exemplarische Umsetzung des Reglers in TwinCAT ist in Abbildung 4.1.3 dargestellt.

Parameter	Wert
KP	1
KI	214
diffConst	0,001

Tabelle 4.1: Parameter des Drehzahlreglers

Schritt	Kurze Erläuterung	TwinCAT Code
1	Bildung des Regelfehlers	<code>errorM1 := targetVelocityMotor1 - actualVelMotor1;</code>
2	Aufsummierung des Integralanteils	<code>IntegralM1 := IntegralM1 + (errorM1 * diffConst);</code>
3	Berechnung des geregelten Ausgangswerts (P- und I-Anteil)	<code>velocityM1Controlled := (errorM1 * KP) + (KI * IntegralM1);</code>
4	Skalierung des Ausgangswerts auf den vom Motor erwarteten Wertebereich	<code>velocityM1Controlled := velocityM1Controlled * skalPos;</code>

Tabelle 4.2: Exemplarischer Ablauf des Drehzahlreglers in TwinCAT

4.2 Positionsregler

Der Positionsregler sorgt dafür, dass eine vorgegebene Sollposition des Roboters korrekt angefahren wird. Für verschiedene Fahrsituationen wird mit einem Gain

Scheduling verschiedenes Fahrverhalten realisiert. Der Positionsregler wird in Twin-CAT als Funktionsbaustein umgesetzt. Als Eingangsgrößen werden Sollpositionen im Weltkoordinatensystem benötigt. Außerdem wird die Istposition im gleichen Koordinatensystem an den Eingang zurückgeführt. Für das Gain Scheduling sind auch die Reglerparameter als Eingangsgrößen veränderbar. Der Ausgang ist die Sollgeschwindigkeit im Roboterkoordinatensystem, welche an den Drehzahlregler aus dem vorherigen Abschnitt weitergeleitet wird. Abbildung 4.3 zeigt den Positionsregler als Blackbox mit Eingangs- und Ausgangssignalen.

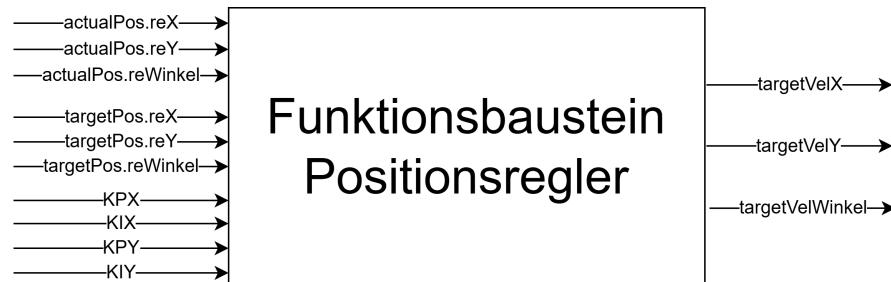


Abbildung 4.3: Positionsregler als Blackbox mit Eingangs- und Ausgangssignalen

4.2.1 Ein- und Ausgangssignale

TargetPosition

Die Target Position ist die Sollposition des Roboters im Weltkoordinatensystem. Sie besteht aus drei Komponenten; der X- und Y- Position und θ -Ausrichtung. Der Positionsregler ist laut Konzept speziell für das Anfahren der Taschen im Nahbereich konzipiert. Somit ist er auf Target Positionen optimiert, welcher sich in der Nähe der aktuellen Position befinden.

ActualPosition

Die Actual Position ist die Istposition des Roboters im Weltkoordinatensystem. Sie besteht ebenfalls aus drei Komponenten; der X- und Y- Position und θ -Ausrichtung. Die Istposition wird von einem Beobachter geschätzt, welcher die aktuelle Position aus den Sensordaten des Roboters berechnet.

KP, KI

Um die Reglerparameter für verschiedene Fahrsituationen anpassen zu können, sind die beiden Parameter KP und KI als Eingangsgrößen veränderbar. Die Parameter werden in einem Gain Scheduling Algorithmus berechnet.

TargetVelocity

Die Target Velocity ist die Sollgeschwindigkeit des Roboters im Koordinatensystem des Roboters. Da der Fehler aus Soll- und Istposition in Weltkoordinaten berechnet wird, muss die Sollgeschwindigkeit in Roboterkoordinaten umgerechnet werden.

4.2.2 Interne Reglerstruktur

Der Positionsregler ist als herkömmlicher P- bzw. PI-Regler ausgelegt. Er bildet aus den beiden Eingangssignalen Sollwert und Istwert einen Regelfehler, welcher über die beiden Reglerpfade proportional und integral verarbeitet wird. Der Regler bedient die drei Freiheitsgrade des Roboters unabhängig voneinander, sodass die Reglerstruktur aus 3 parallelen aber SISO Reglern besteht. Für den Winkel und die Lateralposition - im normalen Betrieb die Y-Position - wird ein P-Regler verwendet. Für die Einfahrtsrichtung - im normalen Betrieb die X-Position - wird ein PI-Regler verwendet. Abbildung 4.4 zeigt die interne Struktur des Positionsreglers für die X-Achse.

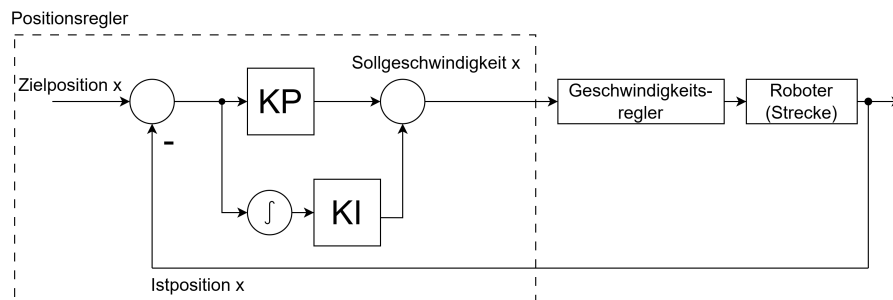


Abbildung 4.4: Interne Struktur des Positionsreglers für die X-Achse

Für die X- und Y-Achse sind die Reglerparameter KP und KI von außen verstellbar. Die Regelung der θ -Achse ist als konstanter P-Regler ohne Integralanteil ausgelegt. Mehr dazu im Abschnitt *Fahralgorithmus mit Gain-Scheduling*. Ein exemplarischer Ablauf des Positionsreglers in TwinCAT ist in Tabelle 4.2.2 dargestellt.

Schritt	Kurze Erläuterung	TwinCAT Code
1	Bildung des Positionsregelfehlers in X-Richtung	<code>error.reX := targetPos.reX - actualPos.reX;</code>
2	Aufsummierung des Integralanteils	<code>integralError.reX := integralError.reX + (error.reX * diffConst);</code>
3	Berechnung der geregelten Sollgeschwindigkeit in X-Richtung (P- und I-Anteil)	<code>W_X_targetSpeed := (error.reX * KPX) + (KIX * integralError.reX);</code>

Tabelle 4.3: Exemplarischer Ablauf des Positionsreglers in TwinCAT

4.2.3 Umrechnung von Welt- und Roboterkoordinaten

Die vom Positionsregler berechnete Sollgeschwindigkeit liegt zunächst im Weltkoordinatensystem vor (`W_X_targetSpeed`, `W_Y_targetSpeed`, `W_Theta_targetSpeed`),

da auch die zugrunde liegende Positionsmessung (`actualPos`) im Weltkoordinatensystem erfolgt. Da der nachgelagerte Drehzahlregler jedoch eine Sollgeschwindigkeit im Koordinatensystem des Roboters (`TargetVelocity`) erwartet, muss die Sollgeschwindigkeit vor der Weitergabe entsprechend der aktuellen Orientierung des Roboters transformiert werden.

Die Transformation erfolgt über eine Rotation um den aktuellen Orientierungswinkel des Roboters θ_{ist} (`actualPos.reWinkel`), welcher die Differenz zwischen der Ausrichtung des Roboters und der des Weltkoordinatensystems angibt. Für die translatorischen Geschwindigkeitskomponenten $\dot{x}$ und $\dot{y}$ ergibt sich damit folgende Drehmatrix:

$$\begin{pmatrix} \dot{x}_{\text{Robot}} \\ \dot{y}_{\text{Robot}} \end{pmatrix} = \begin{pmatrix} \cos(\theta_{\text{ist}}) & -\sin(\theta_{\text{ist}}) \\ \sin(\theta_{\text{ist}}) & \cos(\theta_{\text{ist}}) \end{pmatrix} \begin{pmatrix} \dot{x}_{\text{Welt}} \\ \dot{y}_{\text{Welt}} \end{pmatrix} \quad (4.7)$$

Dabei bezeichnen:

- $\dot{x}_{\text{Welt}}, \dot{y}_{\text{Welt}}$ – vom Positionsregler berechnete Sollgeschwindigkeiten im Weltkoordinatensystem (`W_X_targetSpeed`, `W_Y_targetSpeed`).
- θ_{ist} – aktueller, gemessener Orientierungswinkel des Roboters im Weltkoordinatensystem (`actualPos.reWinkel`).
- $\dot{x}_{\text{Robot}}, \dot{y}_{\text{Robot}}$ – Sollgeschwindigkeiten im Koordinatensystem des Roboters (`R_X_targetSpeed`, `R_Y_targetSpeed`), welche als `TargetVelocity` an den Drehzahlregler weitergegeben werden.

Die Winkelgeschwindigkeit $\dot{\theta}$ (`W_Theta_targetSpeed`) bleibt von dieser Transformation unberührt und wird unverändert als `R_Theta_targetSpeed` übernommen. Dies liegt daran, dass eine Rotation um die Z-Achse unabhängig von der Orientierung des Roboters ist: Die Drehgeschwindigkeit des Roboters um seinen eigenen Mittelpunkt ist in beiden Koordinatensystemen identisch, da sich lediglich die translatorischen Komponenten $\dot{x}$ und $\dot{y}$ durch die unterschiedliche Ausrichtung der Koordinatensysteme unterscheiden.

Somit schließt sich für den Regler ein vierter Schritt an:

Schritt	Kurze Erläuterung	TwinCAT Code
4	Transformation der Sollgeschwindigkeit vom Welt- in das Roboterkoordinatensystem	<pre> R_X_targetSpeed := cos(actualPos.reWinkel) * W_X_targetSpeed - sin(actualPos.reWinkel) * W_Y_targetSpeed; </pre>

Tabelle 4.4: Fortsetzung eines exemplarischen Ablaufs des Positionsreglers in TwinCAT

Anti-Windup Methode

Um zu verhindern, dass sich der Integralanteil der Einfahrtsrichtung unkontrolliert aufsummiert, wird der Integralanteil auf 0 gesetzt, solange $KIX = 0$.

4.2.4 Fahralgorithmus mit Gain-Scheduling

Laut Konzept bringt Gewerk 2 den Roboter in der Vektorfahrweise in die Nähe des Taschenbereichs. Ab dort übernimmt der Positionsregler die Bewegung des Roboters. Bei der Positionsfahrt sollen wesentliche Punkte erfüllt werden: 1. Der Roboter soll die Tasche möglichst schnell und flüssig anfahren. Gleichzeitig darf die Geschwindigkeit beim Andock an die Station nicht zu hoch sein, um das Material nicht unnötig zu belasten. 2. Der Roboter muss die Tasche von vorne anfahren, Da der Greifer für das Werkstück nur von vorne be- und entladen werden kann. 3. Der Roboter muss mit genügend Momentum an die Station fahren, um in der Tasche zu landen und den Slider auszulösen. Dieser Punkt betrifft vorallem einzelne Stationen an denen das Blech der Station etwas nach oben gebogen ist. Hier braucht der Roboter genügend Schwung, um die Station zu überwinden.

Gain Scheduling Algorithmus

Um Punkt 3 zu erfüllen, wurde ein PI-Regler mit Gain-Scheduling in Einfahrtsrichtung gewählt. Prinzipiell würde für einen Positionsregler ein P-Regler ausreichen, da sich ein Integrator in der Strecke befindet und der unterlagerte Geschwindigkeitsregler ebenfalls als PI-Regler ausgelegt ist. Für den Fall, dass die Stellgröße nicht ausreicht, um die verbogene Station zu überwinden, wurde der I-Anteil im Regler hinzugefügt. Um über den gesamten Anfahrweg eine gute Geschwindigkeit zu fahren – also nicht zu langsam, aus Zeitgründen und um die Tasche zuverlässig zu überwinden, und gleichzeitig nicht zu schnell im Stationsbereich, um das Material zu schonen – wird ein Mix aus P- und I-Anteil nach folgendem Gain-Scheduling-Algorithmus berechnet:

Zunächst wird aus der Distanz zur Zielposition Δ_{In} ein normierter Faktor

$$f = \frac{\Delta_{In}}{100}$$

gebildet und auf den Bereich $0 \leq f \leq 1$ begrenzt. Dieser Faktor dient als Interpolationsgröße zwischen den minimalen und maximalen Reglerparametern.

Für den Proportionalanteil gilt

$$K_P = \begin{cases} 0, & \Delta_{In} < 35 \\ K_{P,min} + f (K_{P,max} - K_{P,min}), & \Delta_{In} \geq 35 \end{cases}$$

mit

$$K_{P,min} = 0,1, \quad K_{P,max} = 3,0.$$

Dadurch ist der P-Anteil in unmittelbarer Stationsnähe vollständig deaktiviert. Dies verhindert ein unnötig aggressives Regelverhalten und ermöglicht ein besonders weiches Einfahren in die Zielposition. Mit zunehmender Entfernung steigt der P-Anteil linear an, wodurch eine hohe Dynamik beim Anfahren erreicht wird.

Der Integralanteil wird entgegengesetzt zum P-Anteil angepasst:

$$K_I = \begin{cases} K_{I,\max} - f(K_{I,\max} - K_{I,\min}), & \Delta_{\text{In}} \leq 100 \\ 0, & \Delta_{\text{In}} > 100 \end{cases}$$

mit

$$K_{I,\min} = 0, \quad K_{I,\max} = 0,25.$$

Der I-Anteil besitzt somit in größerer Entfernung von der Station keinen Einfluss auf das Regelverhalten. Erst beim Annähern an die Zielposition nimmt sein Einfluss kontinuierlich zu. Dadurch kann die Regelung stationäre Fehler ausgleichen und eine eventuell erhöhte Reibung oder mechanische Verformungen der Station überwinden. Gleichzeitig bleibt der Regler während des Anfahrens überwiegend proportional ausgelegt, wodurch ein schnelles und stabiles Regelverhalten erreicht wird. Abbildung 4.5 zeigt den Verlauf der Regelparameter beim Gain-Scheduling in Abhängigkeit von der Distanz zur Zielposition.

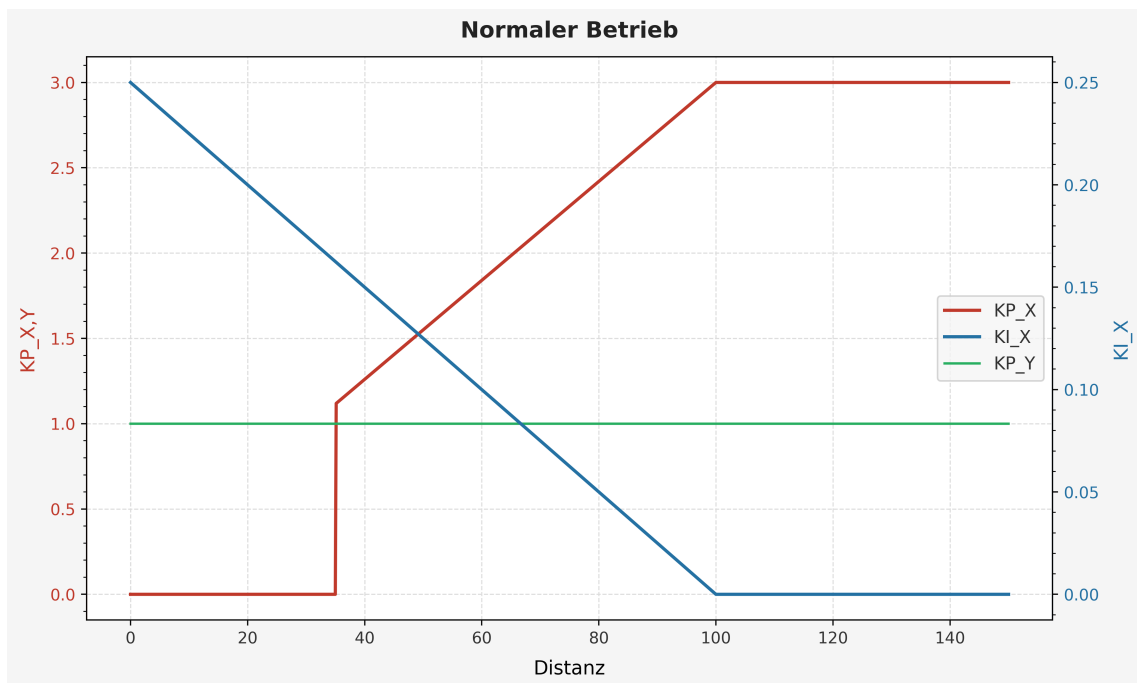


Abbildung 4.5: Abbildung des Gain-Scheduling-Algorithmus für den Positionsregler in Lateral- (konstanter P-Regler, grün) und Einfahrtsrichtung (PI-Regler, rot und blau).

Die gegenläufige Anpassung von P- und I-Anteil vereint somit zwei Anforderungen: Einerseits wird auf langen Fahrstrecken eine hohe Dynamik erzielt, andererseits erfolgt das Einfahren in die Station materialschonend mit hoher Positioniergenauigkeit. **Schrittfolge des Anfahrens** Um Punkt 2 zu erfüllen, könnte man die Bewegungskomponenten in X- und Y-Richtung trennen und nacheinander abarbeiten.

Dies würde jedoch gegen Punkt 1 verstoßen, da die Bewegung in X- und Y-Richtung nicht mehr schnell und flüssig wäre. Um sicherzustellen, dass die Tasche immer von vorne getroffen wird und trotzdem flüssig angefahren wird, wurde eine Schrittkette für das Anfahren mit unterschiedlichen Reglerparametern realisiert:

1. Ausrichtung des Winkels Da die θ -Ausrichtung des Roboters mit in der Drehmatrix steckt, beeinflusst eine Änderung der Drehung auch die Bewegung in X- und Y-Richtung in Weltkoordinaten stark. Das gleichzeitige Fahren und Drehen ist daher schwer umzusetzen, vor allem in Bereichen wo hohe Präzision verlangt wird. Letztlich haben wir uns dazu entschieden die Drehung im Stillstand auszuführen, sobald von Vektor- auf Positionsfahrweise gewechselt wird. Der Winkel wird über einen P-Regler mit konstantem KP geregelt. Das KP beträgt 1,5 und ist bewusst aggressiv gewählt, um möglichst wenig Zeit im Stillstand zu verbringen. Sobald der Winkel in einem bestimmten Bereich liegt, wird die Bewegung in X- und Y-Richtung im nächsten Schritt freigeschaltet

2b. Ausrichtung der Lateralbewegung Die Ausrichtung der Lateralbewegung hat eine höhere Priorität als die Ausrichtung der Vorwärtsbewegung, da die Tasche von vorne angefahren werden muss. Solange sich die Lateralposition noch nicht in einem Toleranzbereich befindet, wird die Vorwärtsbewegung gedrosselt. **2a. Nur Lateralbewegung** Der Fall 2a tritt ein, wenn der Roboter sehr nah an der Station ist, aber die Lateralposition noch nicht in einem Toleranzbereich liegt. In diesem Fall wird die Vorwärtsbewegung komplett gesperrt und nur die Lateralbewegung ausgeführt. Dieser Fall sollte im Normalbetrieb selten auftreten, da die Ausrichtung der Lateralbewegung immer priorisiert wird. Der Fall wird, obwohl er selten auftritt, mit 2a bezeichnet, da er eine höhere Priorität als 2b haben muss.

3. Normale Fahrt nach Gain-Scheduling Algorithmus Sind sowohl Winkel als auch Lateralposition in einem Toleranzbereich, wird die Einfahrt mit dem PI-Regler nach dem Gain-Scheduling Algorithmus ausgeführt. Die Reglerparameter werden über eine Funktion berechnet, welche die Distanz zur Sollposition als Eingangsgröße verwendet.

Abbildung 4.6 zeigt ein Flowchart über die verschiedenen Schritte der Anfahrkette.

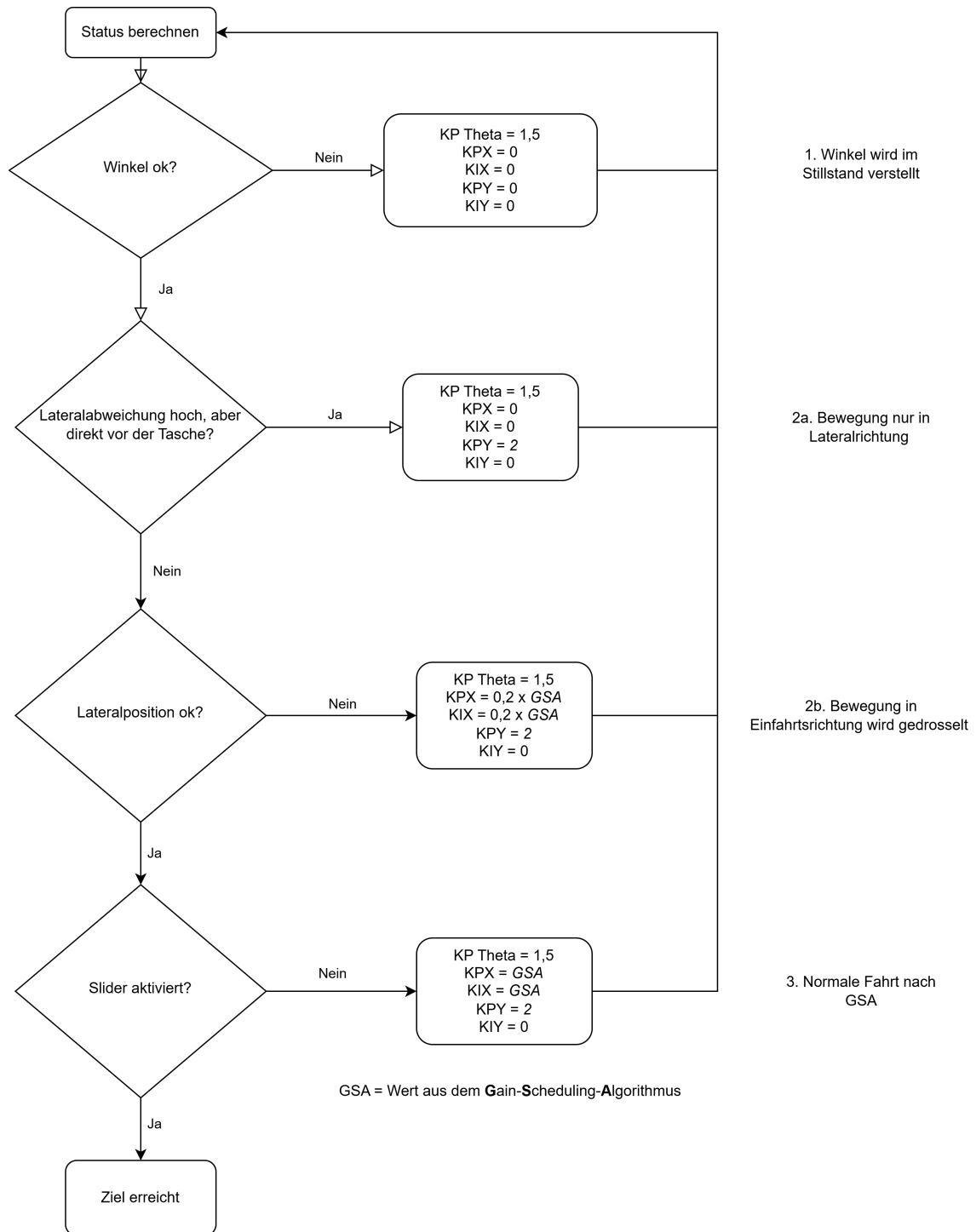


Abbildung 4.6: Flowchart über die Anfahrkette des Positionsreglers mit Gain-Scheduling

Der Gain-Scheduling Algorithmus berechnet und verändert die Reglerparameter nur für die Einfahrtsrichtung, da diese den größten Einfluss auf das Erfüllen der

Anforderungen 1, 2 und 3 hat. Dennoch kann auch KPY und KIY angepasst werden. Der Grund liegt darin, dass die "Tasche" der Ladestation genau andersherum, also aus Y-Richtung angefahren werden muss. Der Roboter weiß durch die Zielposition und -ausrichtung ob es sich um eine Tasche oder eine Ladestation handelt. Für das Anfahren der Ladestation werden alle Regelparameter einfach umgedreht. Somit wird die Bewegung in X-Richtung dann mit einem konstanten P-Regler mit $K_p = 2$ ausgeführt und das Gain Scheduling findet in Y-Richtung statt.

4.3 Auswertung

4.3.1 Vektorfahrweise

Bei der Vektorfahrweise wird nur der Geschwindigkeitsregler benötigt. Er soll die Sollgeschwindigkeit von Gewerk 2 möglichst gut an den Roboter weitergeben. Die Berechnung der Sollgeschwindigkeit erfolgt in Gewerk 2. In der Auswertung zeigt sich, dass der vorgegebene Wert von Gewerk 2 stark schwankt. Abbildung 4.7 zeigt den Sollwert der Geschwindigkeit in der Vektorfahrweise über 200s. Abbildung 4.8 zeigt den entsprechenden Istwert der Geschwindigkeit über dem Sollwert in der Vektorfahrweise über 200s. Da die Sollgeschwindigkeit stark schwankt, ist eine quantitative Auswertung der Reglerperformance schwer möglich. Dennoch ist sowohl in Abbildung 4.8, als auch bei der Vorführung zu sehen, dass die Performance für diese Anwendung gut genug ist.

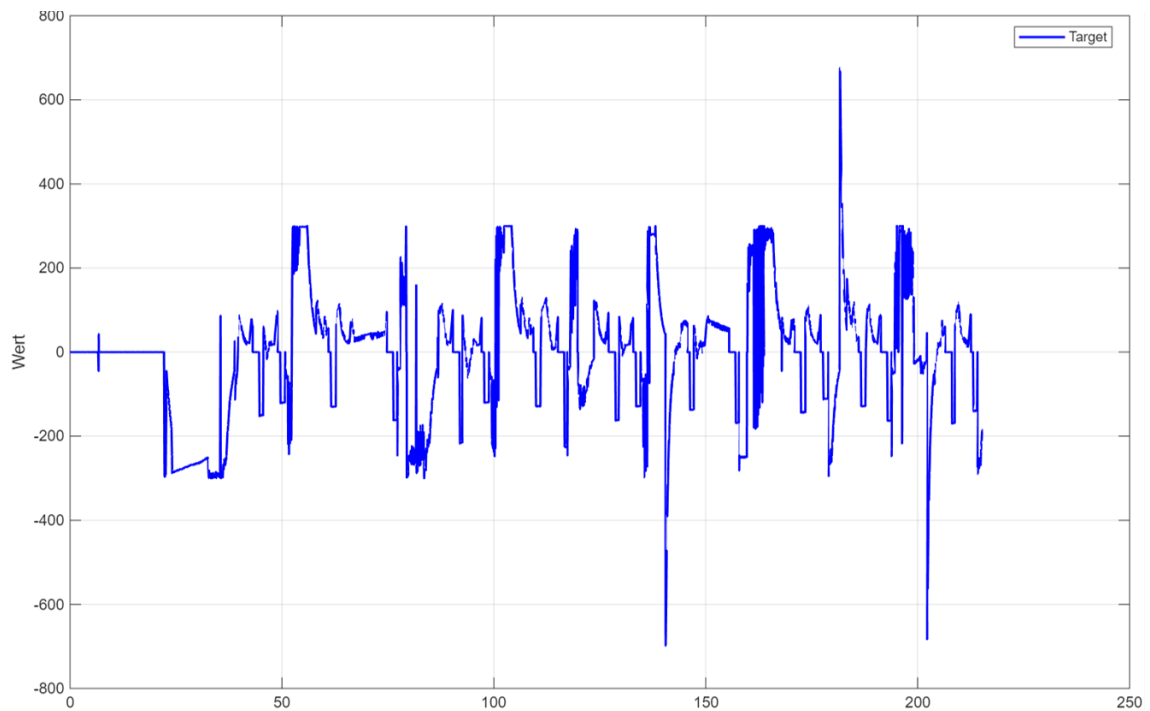


Abbildung 4.7: Sollwert der Geschwindigkeit in der Vektorfahrweise über 200s X-Achse: Sollgeschwindigkeit in mm/s, Y-Achse: Zeit in s

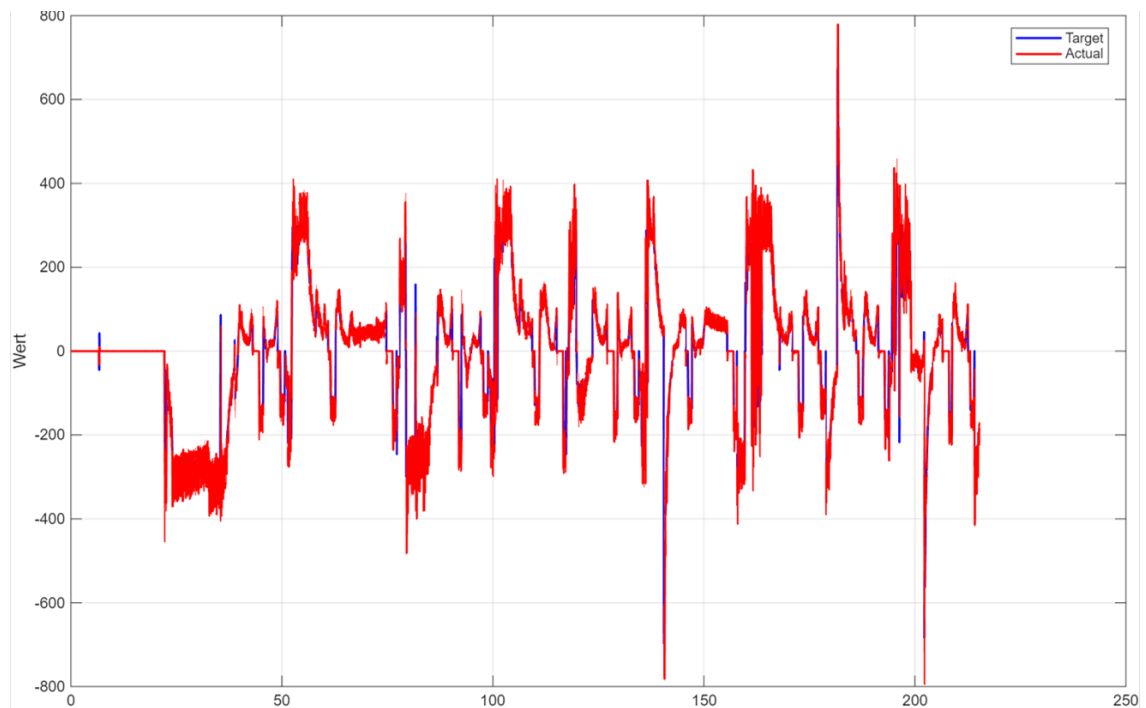


Abbildung 4.8: Istwert der Geschwindigkeit in der Vektorfahrweise über 200s X-Achse: Istgeschwindigkeit in mm/s, Y-Achse: Zeit in s

4.3.2 Positionsfahrweise

Die Positionsfahrweise wurde ebenfalls nur quantitativ und experimentell optimiert. Zentrales Kriterium ist, dass die Lateralausrichtung deutlich schneller erreicht werden soll, als die Einfahrtsrichtung, um Lateralbewegungen im Bereich der Station zu vermeiden. Dies könnte den Greifer beschädigen. Abbildung 4.9 und Abbildung 4.10 gemeinsam zeigen die Anfahrweise einer Station. Nur die Bereiche, die gelb hinterlegt sind, werden mit dem Positionregler ausgeführt. Der Ablauf ist wie folgt:

1. Der Roboter richtet sich nach dem vorgegeben Sollwinkel aus, es findet keine Bewegung in X- oder Y-Richtung statt.
2. Der Roboter fährt die Tasche an und nimmt das Werkstück auf.
3. Gewerk 2 schaltet in Vektorfahrweise, um eine gerade Rückwärtsfahrt zu realisieren.
4. Der RFID-Sensor wird in Positionsfahrweise wie in Schritt 2 angefahren. Eine Neuausrichtung des Winkels ist nicht nötig.
5. Gewerk 2 schaltet wieder in Vektorfahrweise, der Roboter verlässt den Stationsbereich.

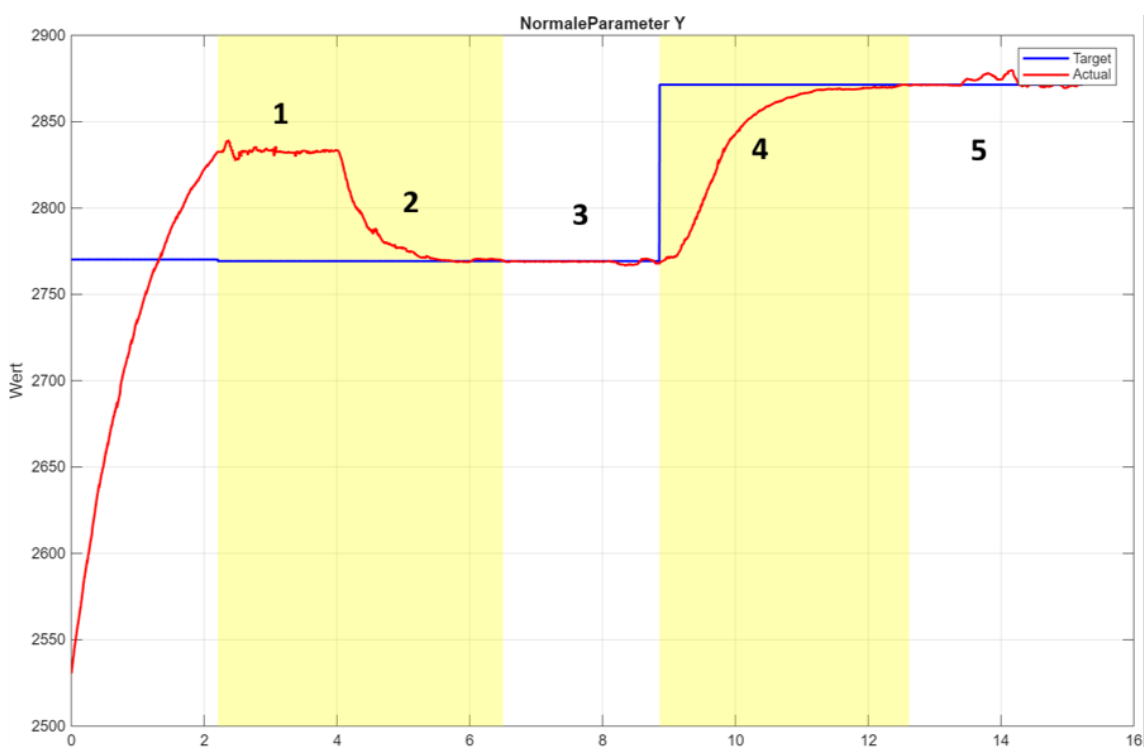


Abbildung 4.9: Anfahrweise einer Station in Y-Richtung

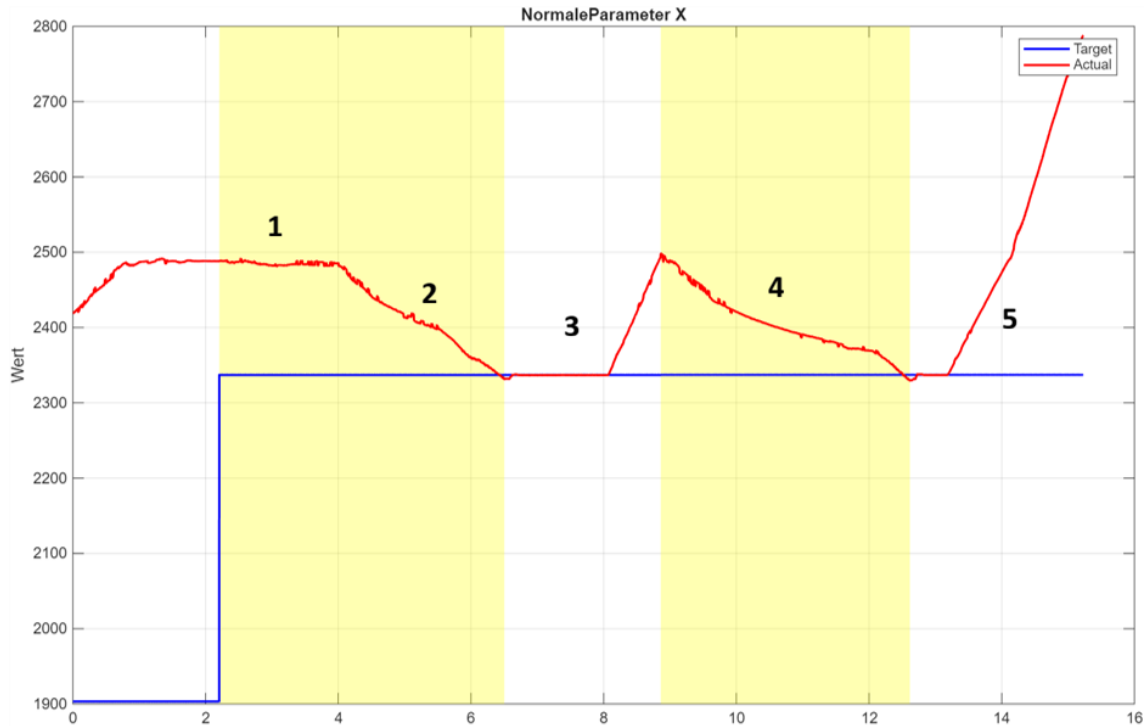


Abbildung 4.10: Anfahrweise einer Station in X-Richtung

Abbildung 4.9 und 4.10 zeigen, dass die entwurfene Reglerstruktur gut geeignet ist um die Anforderungen zu erfüllen. Die Lateralposition wird schnell erreicht und gehalten, die Einfahrtsrichtung ist gemächlicher, um der Lateralausrichtung Zeit zu geben und das Material zu schonen. Das Ausrichten des Winkels ohne X- und Y-Bewegung kostet zwar Zeit, hier haben wir uns aber entschieden dem sicheren Anfahren höhere Priorität als dem schnellen Anfahren einzuräumen. Zwischen Umschalten auf die Positionsfahrweise und Erreichen der Sollposition vergehen mit Drehung etwa 4 Sekunden. Dieser Wert ist im Kontext des Projekts völlig ausreichend und die Fahrweise des Roboters fügt sich auch optisch gut in die Gesamtbewegung ein.

4.4 Entwurf eines Fuzzy-Reglers

4.4.1 Grundlegende Idee und Begründung

Bis dahin fuhr der Roboter auf den langen, geraden Bahnen im Vektorfahrt-Modus. An den einzelnen Stationen angekommen, hielt er kurz an und wechselte anschließend in den Positionsfahrt-Modus. Im Verlauf des Projekts entstand die Idee, zusätzlich einen Fuzzy-Regler zu entwerfen und die Fahrten alternativ auch über diesen zu realisieren. Ein wesentlicher Vorteil bestünde darin, dass der Roboter nicht mehr zwischen den verschiedenen Fahrmodi wechseln müsste. Stattdessen könnte er die Zielpositionen in einer kontinuierlichen, flüssigen Bewegung anfahren, ohne an jeder Position anzuhalten. Darüber hinaus wäre es möglich, seitliche, vorwärts-, rückwärts- und rotierende Bewegungen auszuführen, ohne den Roboter zuvor in Fahrtrichtung ausrichten zu müssen. Da Fuzzy-Regler nicht auf einem exakten linearen Modell des Roboters basieren, sondern mit linguistischen Regeln arbeiten, wurde dieser Regleransatz als sinnvoll erachtet. Anstatt den analytischen Zusammenhang zwischen Regelabweichung und Stellgröße vollständig herzuleiten, wird das Reglerverhalten durch Zugehörigkeitsfunktionen und eine Regelbasis beschrieben. Dadurch kann auf eine aufwendige mathematische Modellierung verzichtet werden. Eine exakte Modellierung des Roboters unter Berücksichtigung von Reibung, Schlupf, Verzögerungen und Sättigungseffekten hätte den Entwicklungsaufwand erheblich erhöht. Ein weiterer Vorteil des Fuzzy-Reglers besteht darin, dass er mit wenigen Parametern relativ einfach an unterschiedliche Anwendungen angepasst werden kann. Daher war es das Ziel, den Regler möglichst allgemein aufzubauen, sodass er später als Baustein für komplexere Fahrmanöver eingesetzt und schnell an verschiedene Anforderungsprofile angepasst werden kann. Dadurch sollte der Regler eine höhere Flexibilität bei zukünftigen Änderungen der Bahnplanung ermöglichen. So könnte beispielsweise die bisherige Vektorfahrt durch eine Punktliste oder einen gleitenden Zielwert ersetzt werden, ohne dass grundlegende Änderungen am Regler erforderlich wären. Darüber hinaus wollte unsere Gruppe praktische Erfahrungen mit Fuzzy-Reglern sammeln. Zwar war bereits theoretisches Wissen zu dieser Reglerstruktur vorhanden, praktische Erfahrungen lagen jedoch noch nicht vor. Das Projekt bot daher die Möglichkeit, diese Wissenslücke in einem kontrollierten Umfeld zu schließen und Erfahrungen im Entwurf sowie in der Implementierung eines Fuzzy-Reglers zu sammeln. Ziel war es daher, einen Fuzzy-Regler zu entwerfen, zu implementieren und seine Eignung für unterschiedliche Fahrarten eines omnidirektionalen Roboters experimentell zu untersuchen. [6, 5, 4]

4.5 Beschreibung Fuzzy-Regler für omnidirektionalen Roboter

4.5.1 Erklärung Fuzzy

Ein klassischer Regler arbeitet mit festen Grenzwerten. Ein Positionsfehler von z.B. 0,10 m führt immer nach einer vorgegebenen Berechnung zu genau einer Stellgröße. Ein Fuzzy-Regler arbeitet dagegen mit mehreren Regelbereichen. Liegt der aktuelle Fehler zwischen zwei Bereichen, wird nicht sprunghaft von einer Regel zur nächsten gewechselt. Stattdessen wird zwischen den beiden zugehörigen Stellgrößen entsprechend der jeweiligen Zugehörigkeit interpoliert. Da zyklisch neu berechnet wird, entstehen stetige Übergänge zwischen den einzelnen Regelbereichen. Die Stellgröße ändert sich kontinuierlich und nicht sprunghaft, wodurch der Roboter ruhiger fährt und ruckartige Bewegungen vermieden werden. Im Projekt wurden für jede Ein- und Ausgangsgröße fünf Regelbereiche definiert: NB, NS, ZE, PS und PB. Diese stehen für Negative Big, Negative Small, Zero, Positive Small und Positive Big. Jedem Bereich ist eine eigene Regel mit einer entsprechenden Stellgröße zugeordnet. Die verwendeten Zugehörigkeitsfunktionen sind dreiecksförmig beziehungsweise an den Randbereichen trapezförmig ausgeführt. Diese Einteilung ist zur Übersicht nochmals in Tabelle 4.5 veranschaulicht. [6]

Tabelle 4.5: Linguistische Zustände des Fuzzy-Reglers

Kürzel	Bedeutung	Interpretation im Regler
NB	Negative Big	starker negativer Fehler oder starke negative Stellgröße
NS	Negative Small	kleiner bis mittlerer negativer Fehler
ZE	Zero	Fehler nahe null; Zielbereich
PS	Positive Small	kleiner bis mittlerer positiver Fehler
PB	Positive Big	starker positiver Fehler oder starke positive Stellgröße

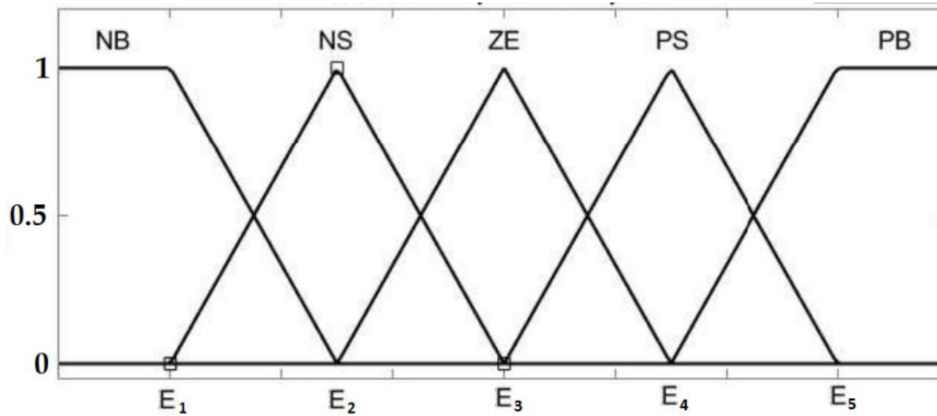


Abbildung 4.11: Schematischer Aufbau von Zugehörigkeitsfunktionen eines Fuzzy-Reglers(übernommen aus [7]).

Abbildung 4.11 zeigt schematisch den Aufbau von Zugehörigkeitsfunktionen eines Fuzzy-Reglers. Auch wenn hier beispielhaft andere Bezeichnungen verwendet werden, entspricht das dargestellte Prinzip dem im Projekt eingesetzten Regler.

4.5.2 Reglerstruktur für x, y und theta

Da sich ein omnidirektionaler Roboter unabhängig in x-Richtung, y-Richtung und um die eigene Achse θ bewegen kann, werden diese drei Bewegungsrichtungen getrennt geregelt. Für jede Richtung wird derselbe Fuzzy-Regler verwendet. Als Eingangsgrößen dienen jeweils der Positionsfehler sowie dessen Änderung. Im fortlaufenden beschrieben als error und derror. Für jede Richtung wird anhand des jeweiligen Positionsfehlers eine Sollgeschwindigkeit berechnet. Diese wird anschließend an die Kinematik übergeben und in Radgeschwindigkeiten umgerechnet. Die verwendeten Ein- und Ausgangsgrößen der drei Regelkanäle sind in Tabelle 4.6 dargestellt. [6]

Tabelle 4.6: Ein- und Ausgangsgrößen der drei Regelkanäle

Regelkanal	Eingänge	Ausgang	Physikalische Bedeutung
x-Regler	$error_x, derror_x$	v_x	Geschwindigkeit in x-Richtung
y-Regler	$error_y, derror_y$	v_y	Geschwindigkeit in y-Richtung
θ -Regler	$error_\theta, derror_\theta$	ω	Drehgeschwindigkeit um die eigene Achse

4.5.3 Regelbasis

Die Regelbasis legt fest, welche Stellgröße sich aus einer bestimmten Kombination von Fehler und Fehleränderung ergibt. Für jeden der drei Regelkanäle wird dieselbe 5×5 -Regelmatrix verwendet. Die Zeilen beschreiben den aktuellen Fehlerzustand, die Spalten dessen Änderung. Das Ergebnis der Regelmatrix ist zunächst ein linguistischer

Ausgangswert. Dieser wird anschließend in eine kontinuierliche numerische Stellgröße umgerechnet, welche direkt als Sollwert für die Bewegung des Roboters verwendet werden kann. Dieser Vorgang wird als Defuzzifizierung bezeichnet. Die verwendete Regelmatrix ist in Tabelle 4.7 dargestellt. [6]

Tabelle 4.7: Regelmatrix des Fuzzy-Reglers

<i>error</i> \ <i>derror</i>	NB	NS	ZE	PS	PB
NB	NB	NB	NB	NS	ZE
NS	NB	NS	NS	ZE	PS
ZE	NS	NS	ZE	PS	PS
PS	NS	ZE	PS	PS	PB
PB	ZE	PS	PB	PB	PB

Die Matrix wurde so gewählt, dass bei großem Fehler zunächst eine starke Stellgröße erzeugt wird, während eine hohe Fehleränderung das Stellverhalten dämpft und dadurch ein weiches Einfahren ermöglicht. Sie ist symmetrisch aufgebaut und sorgt dafür, dass große Fehler zu großen Stellgrößen führen, während Fehler nahe null nur kleine oder keine Stellgrößen erzeugen. Zusätzlich berücksichtigt der Regler die Fehleränderung. Zu Beginn einer Fahrt ist der Positionsfehler zwar groß, die Fehleränderung aufgrund der anfänglich geringen Geschwindigkeit jedoch noch klein. Dadurch wird zunächst keine maximale Stellgröße ausgegeben, sondern der Roboter beschleunigt kontrolliert und ruckarm. Bewegt sich der Roboter bereits in Richtung des Zieles und der Fehler nimmt kontinuierlich ab, reduziert die Regelbasis die Stellgröße frühzeitig. Dadurch wird ein Überspringen vermieden und ein gleichmäßiges sowie ruhiges Einfahren in die Zielposition erreicht.

4.5.4 Skalierung der realen Größen

Der Fuzzy-Regler arbeitet ausschließlich mit normierten Ein- und Ausgangsgrößen im Wertebereich von -1 bis $+1$. Reale Positions- und Winkelfehler können jedoch deutlich größere Werte annehmen und müssen daher vor der Verarbeitung auf diesen Bereich abgebildet werden. Ebenso muss die vom Fuzzy-Regler berechnete normierte Stellgröße anschließend wieder in reale translatorische beziehungsweise rotatorische Geschwindigkeiten umgerechnet werden. Die Skalierung erfolgt über jeweils einen Faktor für die Eingangs- und Ausgangsgrößen. Die Eingangsskalierung bestimmt, welcher reale Fehler einer maximalen Fuzzy-Eingangsgröße entspricht. Eine hohe Skalierung führt dazu, dass bereits kleine Fehler als große Fuzzy-Fehler interpretiert werden, wodurch der Regler aggressiver reagiert. Eine kleinere Skalierung bewirkt dagegen ein ruhigeres Regelverhalten. Die Ausgangsskalierung legt fest, welche reale Geschwindigkeit einer maximalen Fuzzy-Stellgröße entspricht und beeinflusst damit unmittelbar die Dynamik des Roboters. Für die translatorischen Bewegungen in x- und y-Richtung sowie für die Rotation wurden getrennte Skalierungsparameter

verwendet, da sich die erforderlichen Stellgrößen und Regelcharakteristiken deutlich unterscheiden. Die Bestimmung geeigneter Skalierungsparameter erfolgt in Kapitel 4.6 mithilfe einer simulationsbasierten Optimierung.

4.6 Parameterbestimmung durch Python-Simulation

Ziel der Simulation war nicht die Bestimmung vollständig optimaler Reglerparameter, sondern die Ermittlung geeigneter Ausgangswerte für die anschließende Feinabstimmung am realen Roboter. Hierzu wurde eine Simulationsumgebung entwickelt, welche das Regelverhalten des Roboters reproduziert und unterschiedliche Parametersätze anhand einer Kostenfunktion bewertet. Optimierte wurden ausschließlich die sechs Skalierungsparameter des Fuzzy-Reglers. Diese bestimmen die Abbildung zwischen realen Positions- und Winkelfehlern sowie dem normierten Fuzzy-Bereich und legen gleichzeitig fest, wie die berechneten Stellgrößen wieder in reale translatorische und rotatorische Geschwindigkeiten umgesetzt werden.

4.6.1 Zentrale Konfiguration

Alle wesentlichen Einstellungen der Simulation wurden in einer zentralen **CONFIG**-Struktur zusammengefasst. Dadurch können Start- und Zielposition, Simulationsparameter, Robotergeometrie, Kostenfunktionsgewichte sowie Optimierungsparameter an einer Stelle angepasst werden. Dies erleichtert das Testen verschiedener Einstellungen und verbessert gleichzeitig die Übersichtlichkeit des Programms. Diese einzelnen Teile der **CONFIG**-Struktur sind zur Übersicht nochmals in Tabelle 4.8 veranschaulicht.

Tabelle 4.8: Zentrale Konfiguration der Simulation

Bereich	Wichtige Parameter	Bedeutung
start_params	error_scale_xy, derror_scale_xy, output_scale_xy, error_scale_theta, derror_scale_theta, output_scale_theta	Startwerte der zu optimierenden Reglerparameter
pose	x_start, y_start, theta_start, x_target, y_target, theta_target	Start- und Zielposition der Simulation
simulation	dt, steps, v_limit, omega_limit, pos_tol, theta_tol	Zeitschritt, Simulationsdauer und Abbruchbedingungen
robot	robot_radius, wheel_diameter, wheel_linear_limit, wheel_angles_deg	Geometrie und Begrenzungen des omnidirektionalen Roboters
cost_weights	pos_weight, theta_weight, time_weight, smooth_*	Gewichtung der Kostenfunktion
optimization	maxiter, popsize, tol, seed, polish	Einstellungen des Optimierers

4.6.2 Skalierung

Für die Simulation wurden die in Kapitel 4.6 beschriebenen Skalierungsparameter verwendet. Die Positionsfehler werden zunächst auf einen Bereich von -1 m bis 1 m normiert und anschließend mithilfe der Skalierungsparameter auf den Fuzzy-Eingangsbereich abgebildet. Für den Winkelfehler erfolgt dieselbe Vorgehensweise mit einem Bereich von $-\pi$ bis $+\pi$. Zur Einhaltung der physikalischen Grenzen werden die Sollgeschwindigkeiten zusätzlich begrenzt. Für die translatorische Bewegung wurde eine maximale Geschwindigkeit von $0,94\text{ m s}^{-1}$ verwendet. Die maximal zulässige Winkelgeschwindigkeit ergibt sich aus der maximalen Umfangsgeschwindigkeit der Räder und dem Abstand der Räder zum Mittelpunkt des Roboters:

$$v_{\text{Rad}} = r \cdot \omega \quad (4.8)$$

Daraus folgt:

$$\omega_{\text{max}} = \frac{v_{\text{Rad,max}}}{r} = \frac{0,94\text{ m s}^{-1}}{0,15\text{ m}} \approx 6,27\text{ rad s}^{-1}. \quad (4.9)$$

Diese Werte dienen als obere Begrenzung der Sollgeschwindigkeiten innerhalb der Simulation und stellen sicher, dass keine physikalisch nicht realisierbaren Stellgrößen erzeugt werden.

4.6.3 Simulationsmodell und Ablauf

Die Simulation bildet den vollständigen Regelkreis des Roboters nach und dient als Grundlage für die Bewertung unterschiedlicher Parametersätze. Hierzu werden drei Instanzen des in Kapitel 2 ODERSO beschriebenen Fuzzy-Reglers für die Bewegungen in x-Richtung, y-Richtung sowie für die Orientierung verwendet. Zu Beginn jedes Simulationsschritts werden aus der aktuellen Positions- und Winkelfehler sowie deren Änderungen bestimmt. Diese Größen werden dem Fuzzy-Regler übergeben, welcher daraus translatorische und rotatorische Sollgeschwindigkeiten berechnet. Anschließend werden die Sollgeschwindigkeiten mithilfe der inversen Kinematik in Radgeschwindigkeiten umgerechnet. Überschreiten diese die zulässigen Grenzwerte des Roboters, erfolgt eine Begrenzung auf die maximal realisierbaren Radgeschwindigkeiten. Aus den begrenzten Radgeschwindigkeiten werden anschließend über die Vorwärtskinematik die tatsächlich erreichbaren translatorischen und rotatorischen Geschwindigkeiten bestimmt. Mit diesen Geschwindigkeiten wird die Roboterpose für den nächsten Simulationsschritt berechnet. Dieser Ablauf wird solange wiederholt, bis die Zielpose innerhalb der vorgegebenen Positions- und Orientierungstoleranzen erreicht ist oder die maximale Simulationsdauer überschritten wird.

4.6.4 Kostenfunktion

Die Qualität eines Parametersatzes wird über eine Kostenfunktion bewertet. Dabei werden sowohl die Genauigkeit der Zielerreichung als auch das dynamische Verhalten des Roboters berücksichtigt. Die Kostenfunktion setzt sich aus Positionskosten, Winkelkosten, Zeitkosten, Stellgrößenkosten, Glattheitskosten sowie einer Strafkomponekte zusammen, falls die Zielposition innerhalb der vorgegebenen Toleranzen nicht erreicht wird. Die Einzelnen Kostenfaktoren und deren Bedeutung sind nochmals in Tabelle 4.9 dargestellt.

Tabelle 4.9: Bestandteile der Kostenfunktion

Kostenanteil	Bedeutung
Positionskosten	Bewerten die verbleibende Positionsabweichung am Ende der Simulation.
Winkelkosten	Bewerten die verbleibende Orientierungsabweichung.
Zeitkosten	Bevorzugen kürzere Fahrzeiten beziehungsweise weniger Simulationsschritte.
Stellgrößenkosten	Bestrafen hohe translatorische und rotatorische Stellgrößen.
Glattheitskosten	Bestrafen Sprünge in v_x , v_y und ω und reduzieren damit Oszillationen.
Penalty	Hohe Zusatzkosten, wenn das Ziel innerhalb der Toleranzen nicht erreicht wird.

4.6.5 Optimierungsverfahren

Zur Optimierung der sechs Skalierungsparameter wurde das Verfahren `differential_evolution` aus der Bibliothek `scipy.optimize` verwendet. Aufgrund seiner Robustheit und der guten Dokumentation wurde es für diese Arbeit ausgewählt.

4.6.6 Modellannahmen und Einschränkungen

Zur Begrenzung des Rechenaufwands wurden bei der Simulation bewusst mehrere Vereinfachungen vorgenommen. Das Robotermodell berücksichtigt ausschließlich die Kinematik des Fahrzeugs. Dynamische Effekte wie Massenträgheiten, Reibung, Radschlupf oder das Verhalten der Motoren werden nicht modelliert. Eine dynamische Modellierung erhöht die Genauigkeit stärker bei hohen Geschwindigkeiten und Beschleunigungen. Da die Optimierung jedoch lediglich geeignete Startparameter für die anschließende Feinabstimmung liefern sollte und die höchste Genauigkeit vor allem beim langsamen Anfahren der Zielposition erforderlich ist, wurde diese Vereinfachung als sinnvoll angesehen. Darüber hinaus wurden ausschließlich die sechs Skalierungsparameter optimiert. Die Regelbasis sowie die Membership-Funktionen blieben unverändert, da deren zusätzliche Optimierung die Rechenzeit deutlich erhöht hätte. Außerdem erfolgte die Optimierung anhand einer einzelnen Referenzfahrt. Eine Bewertung über verschiedene Fahrprofile hätte robustere Parametersätze geliefert, den Rechenaufwand jedoch nahezu proportional zur Anzahl der zusätzlichen Simulationen erhöht. Bereits unter diesen Randbedingungen betrug die Rechenzeit der Optimierung mehr als sieben Stunden. Die getroffenen Vereinfachungen stellen daher einen sinnvollen Kompromiss zwischen Modellgenauigkeit und Rechenaufwand dar. [5, 4]

4.6.7 Ergebnis der Simulation

Die Simulation mit den Startparametern erreicht das Ziel bereits, zeigt aber eine höhere Gesamtkostenbewertung als der optimierte Parametersatz. Abbildung 4.12 zeigt die Membership-Funktionen, Trajektorie, Winkelverlauf, Fehlerverläufe und Stellgrößen der simulierten Fahrt mit den Startwerten und der optimierten Fahrt.

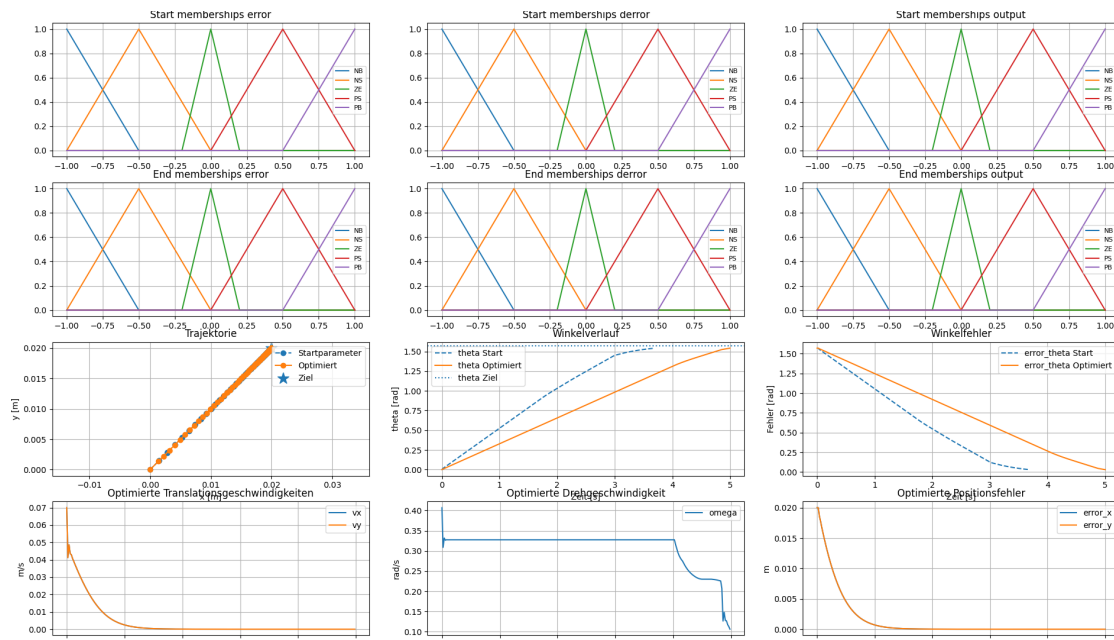


Abbildung 4.12: Gesamtauswertung der Simulation mit Membership-Funktionen, Trajektorie, Winkelverlauf, Fehlern und Stellgrößen.

4.6.8 Numerische Ergebnisse

Die Ergebnisse wurden per Konsolenausgabe dokumentiert und umfassen die Kosten, Endwerte und optimierten Parameter. Für die Startparameter ergibt sich eine Gesamtkostenbewertung von etwa 4,078445. Nach der Optimierung sinken die Kosten auf etwa 2,711807. Die ermittelten Werte im Vergleich zu den gesetzten Simulationsstartwerten sind in Tabelle 4.10 dargestellt.

Tabelle 4.10: Vergleich der Startparameter und optimierten Skalierungsparameter

Parameter	Start	Optimiert	Interpretation
error_scale_xy	2,000000	4,445558	Positionsfehler in x/y werden im Fuzzy-Bereich stärker gewichtet.
derror_scale_xy	0,500000	0,335655	Fehleränderungen in x/y werden etwas schwächer gewichtet.
output_scale_xy	0,500000	0,266333	Maximale translatorische Ausgangsgeschwindigkeit wird reduziert.
error_scale_theta	1,500000	3,962339	Winkelfehler wird deutlich empfindlicher bewertet.
derror_scale_theta	0,300000	0,242722	Änderung des Winkelfehlers wird leicht schwächer gewichtet.
output_scale_theta	1,000000	0,488234	Drehgeschwindigkeit wird etwa halbiert.

4.6.9 Interpretation der optimierten Skalierung

Die optimierten Parameter zeigen ein klares Muster. Die Eingangsskalierung für Positions- und Winkelfehler wird erhöht. Dadurch reagiert der Regler bereits bei kleineren realen Fehlern deutlicher. Gleichzeitig werden die Ausgangsskalierungen reduziert. Das bedeutet, dass der Regler zwar früh in stärkere Fuzzy-Bereiche gelangt, die reale Geschwindigkeit aber begrenzt bleibt. Dieses Verhalten kann zu einer feineren Annäherung an das Ziel führen, weil die Stellgröße nicht zu groß wird. Die Abbildung 4.13 zur Skalierung macht diesen Zusammenhang sichtbar. Die optimierte `error_xy`-Skalierung erreicht den normierten Sättigungsbereich schneller als die Startskalierung. Bei `output_xy` ist die optimierte reale Geschwindigkeit dagegen kleiner. Für θ ist ein ähnlicher Effekt zu erkennen: Der Winkelfehler wird empfindlicher in den Fuzzy-Bereich abgebildet, während die reale Drehgeschwindigkeit reduziert wird.

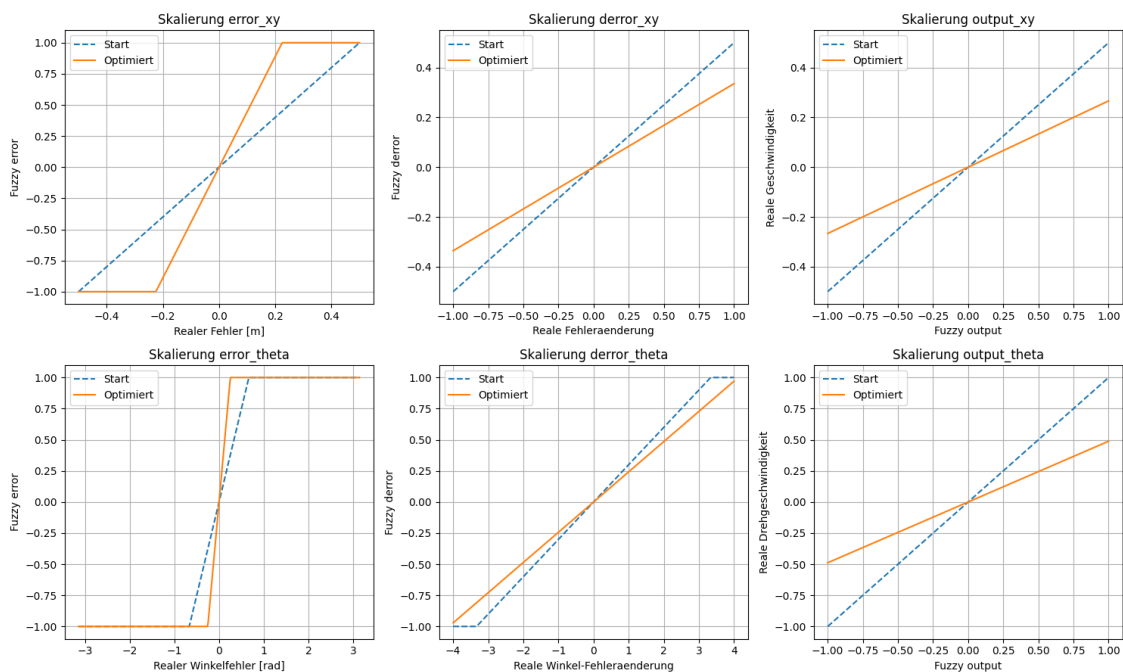


Abbildung 4.13: Vergleich der Start- und Endskalierungen für Positionsfehler, Fehleränderung und Ausgangsgrößen.

4.6.10 Bewertung der Simulation

Die Simulation liefert einen strukturierten ersten Parametersatz und zeigt, dass der Regler grundsätzlich funktioniert. Gleichzeitig ist sie bewusst einfach gehalten. Sie enthält keine vollständige Modellierung von Haft- und Gleitreibung, Radschlupf, Motorverzögerungen, Kommunikationslatenzen oder Messrauschens. Deshalb darf das Simulationsergebnis nicht direkt als endgültige Parametrierung verstanden werden. Es ist vielmehr ein Startpunkt für die reale Anpassung. Besonders kritisch ist der

Unterschied zwischen idealer Kinematik und realer Fahrmechanik. In der Simulation wird angenommen, dass die berechneten Radgeschwindigkeiten direkt umgesetzt werden.

4.7 Umsetzung des Fuzzy-Reglers in der Robotersteuerung

4.7.1 Integration in die bestehende Steuerungsstruktur

Der in Kapitel 3 entwickelte Fuzzy-Regler wurde in die bestehende Robotersteuerung unter TwinCAT integriert. Dabei ersetzt er ausschließlich die Berechnung der Sollgeschwindigkeiten innerhalb der Positionsregelung. Die darunterliegende Geschwindigkeitsregelung der einzelnen Motoren sowie die inverse Kinematik bleiben unverändert bestehen. Die zentrale Ablaufsteuerung erfolgt innerhalb der `MainMotorTask`. Dort werden die aktuelle Roboterposition sowie die Sollposition eingelesen und an den Funktionsbaustein `FB_Positionsregler` übergeben. Dieser berechnet aus den aktuellen Roboter- und Sollwerten die translatorischen Sollgeschwindigkeiten in x- und y-Richtung sowie die rotatorische Sollgeschwindigkeit. Da diese Geschwindigkeiten zunächst im Weltkoordinatensystem vorliegen, werden sie anschließend in das Roboterkoordinatensystem transformiert und an die inverse Kinematik übergeben. Dort werden die Sollgeschwindigkeiten der einzelnen Räder berechnet und als Führungsgrößen an die bestehenden Geschwindigkeitsregler der Motoren weitergegeben. Durch diese Struktur beschränkten sich die Änderungen an der Software auf den Bereich der Positionsregelung. Die vorhandene Motorregelung sowie die Hardwareansteuerung konnten unverändert übernommen werden, wodurch sich der Fuzzy-Regler ohne grundlegende Änderungen in die bestehende Softwarearchitektur integrieren ließ.

4.7.2 Praktische Erweiterungen

Bei der Übertragung des simulierten Fuzzy-Reglers auf den realen Roboter zeigte sich, dass zusätzliche Maßnahmen erforderlich sind, um unter realen Einsatzbedingungen ein stabiles und robustes Regelverhalten sicherzustellen. Hierzu wurden insbesondere die Zielerkennung erweitert sowie die Verarbeitung der Kameradaten angepasst. Zur sicheren Erkennung der Zielposition wurde im `FB_Positionsregler` eine Hysterese implementiert. Ohne diese würde der Roboter bereits bei kleinsten Positionsänderungen oder Messabweichungen fortlaufend zwischen den Zuständen „Ziel erreicht“ und „Ziel nicht erreicht“ wechseln und dadurch unnötige Korrekturbewegungen ausführen. Durch unterschiedliche Schwellwerte für das Erreichen und Verlassen des Zielbereichs verbleibt der Roboter innerhalb einer definierten Toleranz stabil im Endzustand. Eine weitere Anpassung betrifft die Verarbeitung der Kameramessungen. Die Zykluszeit der Robotersteuerung ist deutlich kleiner als die Aktualisierungsrate der Kamera, sodass derselbe Messwert häufig über mehrere Regelzyklen hinweg anliegt. Würde dieser in jedem Zyklus erneut verarbeitet, ergäbe sich über mehrere Zyklen hinweg keine

Fehleränderung. Beim Eintreffen einer neuen Kameramessung entstünde dadurch eine sprunghafte Änderung, welche das Regelverhalten negativ beeinflussen könnte. Aus diesem Grund wird zunächst geprüft, ob tatsächlich eine neue Kameramessung vorliegt. Nur in diesem Fall wird die Fehleränderung aktualisiert. Liegt kein neuer Kamerawert vor, wird die Roboterpose mithilfe der Motordaten fortgeschrieben. Befindet sich der Roboter bereits innerhalb des Zielbereichs, bleibt dieser Zustand unabhängig davon erhalten, sodass die Zielposition weiterhin zuverlässig erkannt wird. Durch diese Ergänzungen konnte der in der Simulation entwickelte Regler erfolgreich auf das reale System übertragen werden. Insbesondere die Hysterese sowie die angepasste Verarbeitung der Kameradaten tragen zu einem ruhigen und stabilen Fahrverhalten des Roboters bei.

4.8 Test am realen System, Ergebnisse und Erkenntnisse

Nach der simulationsbasierten Parametrierung wurde das Regelverhalten am realen System untersucht. Zunächst wurde die grundsätzliche Funktion des Fuzzy-Reglers anhand einfacher Fahrmanöver überprüft, um die korrekte Integration in die Robotersteuerung sicherzustellen. Anschließend erfolgte die Erprobung unter realen Einsatzbedingungen. Hierbei wurden verschiedene Zielpositionen und Orientierungen angefahren und das Fahrverhalten hinsichtlich bewertet. Besonderes Augenmerk lag dabei auf einem ruhigen Anfahrverhalten, dem Vermeiden von Überschwüngen sowie einem gleichmäßigen und stabilen Einfahren in die Zielposition. Während der Versuche wurden die Daten der drei Antriebsmotoren aufgezeichnet. Mithilfe der Vorwärtskinematik wurde aus diesen Messdaten die tatsächlich zurückgelegte Trajektorie des Roboters rekonstruiert und anschließend grafisch dargestellt. Die Auswertung dieser Fahrten diente sowohl der Beurteilung der Regelgüte als auch der Feinabstimmung der zuvor in der Simulation ermittelten Reglerparameter.

4.8.1 Positionsfahrt im realen System

Zur Überprüfung der Positionsregelung wurde eine beispielhafte Fahrt mit mehreren Zielpositionen durchgeführt. Ausgehend von einer beliebigen Startposition wurden vier Zielpunkte im Arbeitsraum angefahren. Zwischen den einzelnen Zielpunkten wurden sowohl translatorische Bewegungen als auch Änderungen der Roboterorientierung durchgeführt. Am zweiten Zielpunkt wurde ausschließlich die Sollorientierung geändert, während die Position unverändert blieb. Abbildung 4.14 zeigt die rekonstruierte Trajektorie des Roboters in der XY-Ebene. Die schwarze Linie stellt die berechnete Fahrbahn dar, während die roten Kreuze die vorgegebenen Zielpositionen markieren. Zusätzlich sind die Sollorientierungen an den einzelnen Zielpunkten sowie die während der Fahrt gemessene Orientierung des Roboters durch blaue Pfeile dargestellt.

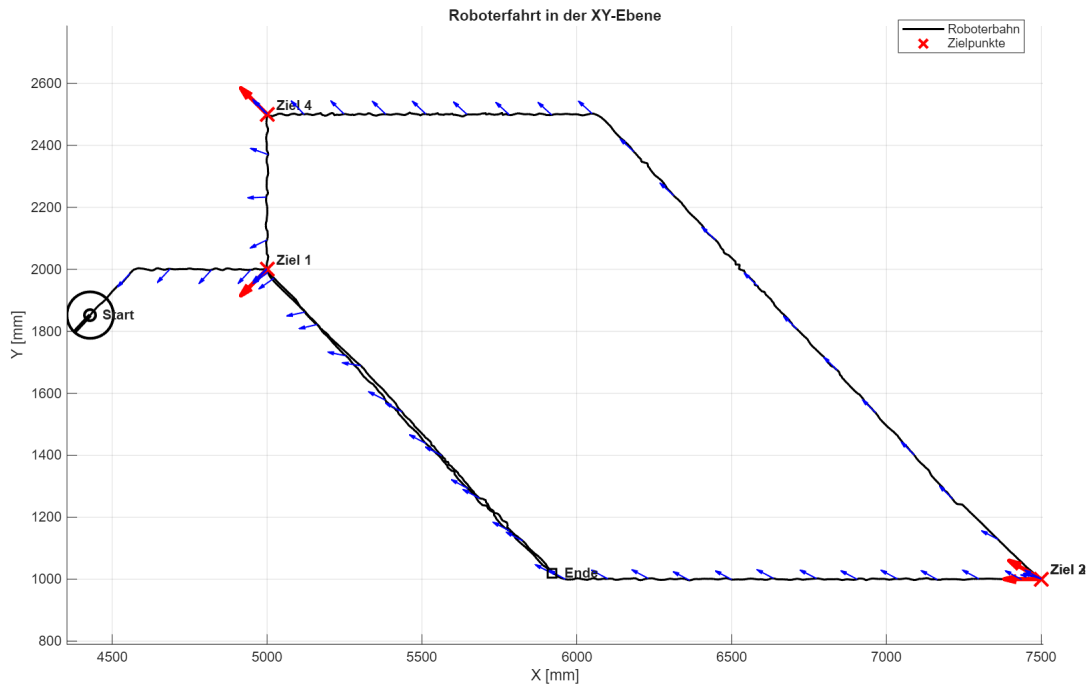


Abbildung 4.14: Rekonstruierte Roboterfahrt in der XY-Ebene mit Start, Ende und Zielpunkten.

Die Messergebnisse zeigen, dass alle Zielpositionen zuverlässig erreicht werden. Auch die Orientierungsänderungen werden während der Fahrt korrekt ausgeführt. Die rekonstruierten Fahrbahnen verlaufen weitgehend entlang der geplanten Bewegungsrichtungen und bestätigen die korrekte Umsetzung des Fuzzy-Reglers auf dem realen System. Zur besseren Veranschaulichung der einzelnen Bewegungsabschnitte ist in Abbildung 4.15 exemplarisch der Verlauf der y-Position über der Zeit dargestellt. Die einzelnen Plateaus entsprechen den angefahrenen Zielpositionen, während die linearen Übergänge die Bewegungen zwischen den einzelnen Punkten zeigen. Die Abbildung verdeutlicht, dass die Zielpositionen stabil erreicht werden und verhart bis die anderen Position, z.B. in X-Richtung, erreicht sind.

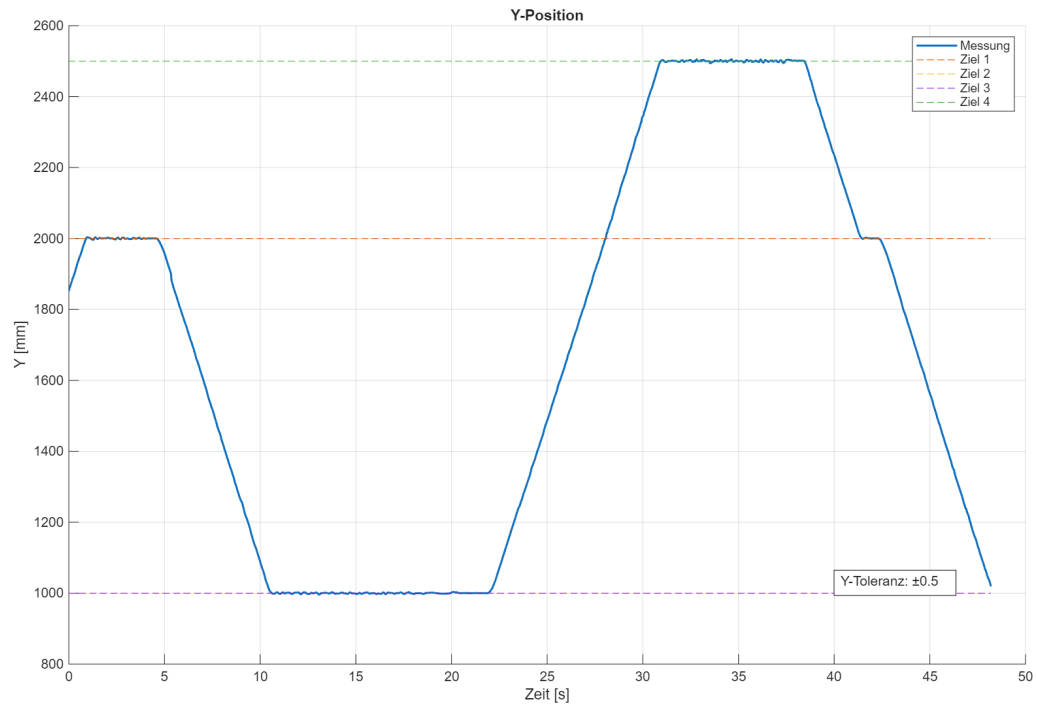


Abbildung 4.15: Rekonstruierte Roboterfahrt in der y-Richtung.

4.8.2 Anfahrt des Ablagepunkts

Nach der grundsätzlichen Verifikation der Positionsregelung wurde das Verhalten des Roboters an einem für den praktischen Einsatz besonders relevanten Ablagepunkt untersucht. Für diesen Zielpunkt galten deutlich engere Positions- und Orientierungstoleranzen, da ein präzises Anfahren erforderlich ist, um einen sicheren Ablagevorgang zu gewährleisten. Wie bereits im vorherigen Versuch wurden während der Fahrt die Drehgeberdaten der drei Antriebsmotoren aufgezeichnet und mithilfe der Vorwärtskinematik die tatsächlich zurückgelegte Trajektorie rekonstruiert. Ziel 2 stellt hierbei den Ablagepunkt dar und wird mit den erhöhten Genauigkeitsanforderungen angefahren.

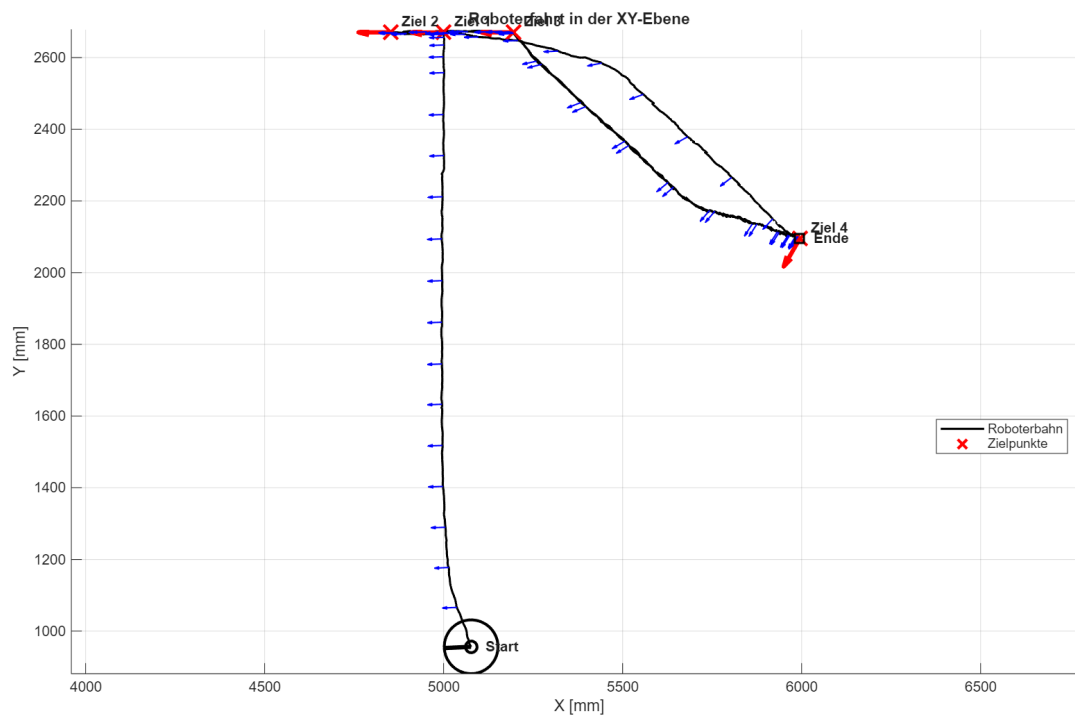


Abbildung 4.16: Rekonstruierte Roboterfahrt in der XY-Ebene mit Richtungspfeilen.

Abbildung 4.16 zeigt die rekonstruierte Roboterfahrt in der XY-Ebene. Die Darstellung ist Analog wie die in Abbildung 4.14. Zur Veranschaulichung der Positionsgenauigkeit ist in Abbildung 4.17 zusätzlich der Verlauf der x-Position über der Zeit dargestellt. Die eingezeichneten Sollpositionen sowie die zulässigen Toleranzbereiche verdeutlichen, dass der Roboter den Ablagepunkt innerhalb der geforderten Genauigkeit erreicht und stabil hält.

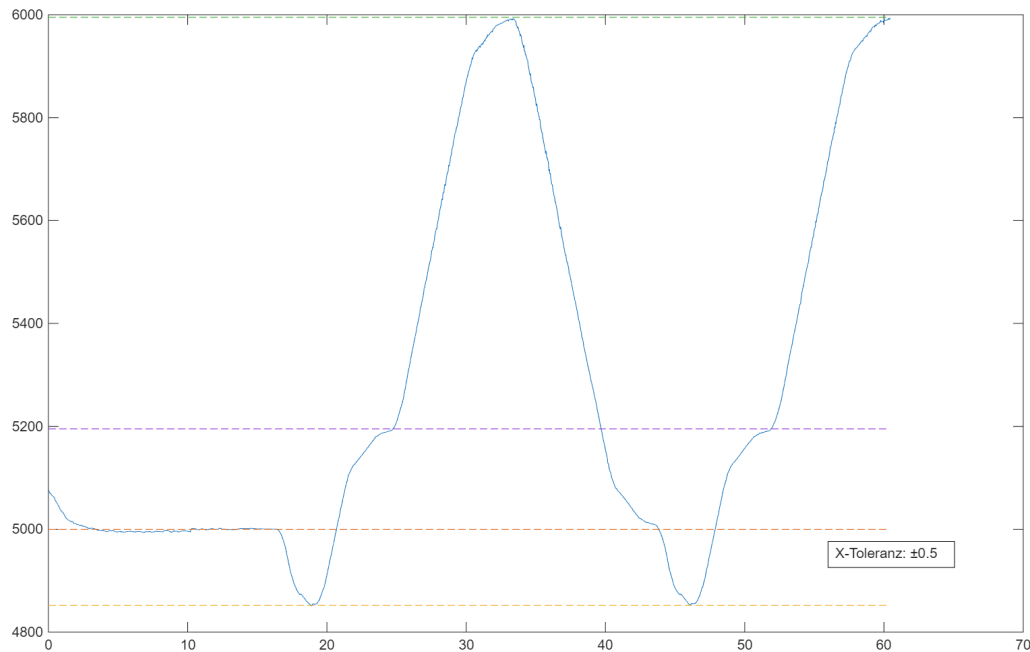


Abbildung 4.17: Weiterer X-Verlauf einer aufgezeichneten Fahrt mit Zielniveaus und Toleranzhinweis.

Die Versuchsergebnisse zeigen, dass der entwickelte Fuzzy-Regler auch bei erhöhten Genauigkeitsanforderungen zuverlässig arbeitet. Der Ablagepunkt wurde innerhalb der vorgegebenen Positions- und Orientierungstoleranzen angefahren. Während der Versuche traten weder Kollisionen noch instabile Regelvorgänge auf. Damit konnte nachgewiesen werden, dass sich der entwickelte Regler sowohl für allgemeine Positionsfahrten als auch für präzise Positionieraufgaben im praktischen Einsatz eignet.

4.9 Vergleich zwischen Simulation und realem System

Die im Rahmen der Simulation ermittelten Reglerparameter stellten eine sehr gute Ausgangsbasis für die Inbetriebnahme des realen Systems dar. Dennoch zeigte sich, dass eine geringfügige Nachjustierung einzelner Skalierungsparameter erforderlich war. Der wichtigste Unterschied zwischen Simulation und realem System liegt in der Modelltreue. Während in der Simulation ausschließlich das kinematische Verhalten des Roboters berücksichtigt wird, treten im realen System zusätzlich dynamische Effekte wie Massenträgheit, Reibung, Radschlupf, mechanische Toleranzen sowie Messrauschen auf. Insbesondere im Bereich der Ablageposition spielt außerdem die vorhandene Federung eine wichtige Rolle. Um den Zielpunkt zuverlässig zu erreichen, muss der Roboter mit einer geringen Restgeschwindigkeit in die federnd gelagerte Position einfahren. Dieses Verhalten wird im verwendeten Simulationsmodell nicht berücksichtigt und machte daher eine Feinabstimmung der Parameter am realen

System erforderlich. Insgesamt zeigte sich jedoch, dass die Simulation ihren Zweck erfüllt hat.

4.10 Fazit und Ausblick

Im Rahmen dieser Arbeit wurde ein Fuzzy-Regler für einen omnidirektionalen Roboter entwickelt, simuliert und erfolgreich in die bestehende Robotersteuerung integriert. Ziel war es, eine einheitliche Reglerstruktur zu schaffen, die sowohl für Positionsfahrten als auch als Grundlage für zukünftige Vektorfahrten verwendet werden kann. Die Simulation ermöglichte eine automatische Bestimmung geeigneter Skalierungsparameter und zeigte sich als gut geeignet. Die anschließenden Fahrversuche zeigten, dass der Regler Zielpositionen und Zielorientierungen zuverlässig anfährt und auch bei engen Toleranzen ein stabiles Regelverhalten erreicht. Kleinere Anpassungen der Simulationsparameter waren aufgrund realer Einflüsse wie mechanischer Toleranzen und der Federung im Bereich der Ablageposition erforderlich, änderten jedoch nichts am grundsätzlichen Erfolg des entwickelten Regelungskonzepts. Besonders vorteilhaft erwiesen sich der modulare Aufbau des Reglers, die übersichtliche Regelbasis sowie die Möglichkeit, dieselbe Reglerstruktur sowohl für Positions- als auch für Vektorfahrten zu verwenden. Die Parametrierung erfolgt ausschließlich über wenige Skalierungsparameter und kann dadurch einfach an unterschiedliche Anforderungen angepasst werden. Insgesamt konnten die zu Beginn formulierten Ziele erreicht werden. Der entwickelte Fuzzy-Regler stellt eine robuste und gut nachvollziehbare Lösung für die Regelung des omnidirektionalen Roboters dar und bildet eine geeignete Grundlage für zukünftige Erweiterungen. Für zukünftige Arbeiten bietet sich insbesondere eine Erweiterung der Simulation um ein dynamisches Robotermodell an. Darüber hinaus könnten weitere Reglerparameter, beispielsweise die Zugehörigkeitsfunktionen oder die Regelbasis selbst, automatisch optimiert sowie unterschiedliche Fahrszenarien gleichzeitig zur Parametrierung herangezogen werden. Ebenso stellt die Umsetzung einer kontinuierlichen Vektorfahrt durch gleitende Zielpunkte einen vielversprechenden nächsten Entwicklungsschritt dar.

5 Fazit

- Noch Bumper, Abstandssensoren und Problem mit Codebasis ansprechen (Alte Leichen in den GVLs)
- Probleme mit anderen Gewerken ansprechen (Kommunikation, verlässlichkeit, Hebel)
- Arbeit über Semesterferien früher kommunizieren. War am Anfang nicht klar.
- Grundsätzlich aber viel mitgenommen, Freiheiten sehr genossen.

Abbildungsverzeichnis

2.1	Blockschaltbild der Positionregelung	4
2.2	Blockschaltbild der Geschwindigkeitsregelung	4
3.1	Vergleich der gefilterten Geschwindigkeitsverläufe aus Motordrehzahlen und Kamerapositionen.	9
3.2	Positionsverläufe des Roboters bei offener Ansteuerung ohne Positionsregelung.	10
3.3	Vergleich der ermittelten Nullstellen bei der Geschwindigkeitssignale zur Bestimmung der Systemlatenz.	11
3.4	3-Rad-Omniroboter mit Radwinkeln 60°, 180°, 300°	12
3.5	Zustandsbeobachter nach Luneburger [3]	14
3.6	Konzept Kalman-Filter [1]	15
3.7	Beobachter mit direktem Ausgleich Normalfahrt	17
3.8	Beobachter mit direktem Ausgleich Normalfahrt Verschoben	18
3.9	Beobachter mit direktem Ausgleich Gesamt	19
3.10	Beobachter mit direktem Ausgleich	20
3.11	Beobachter mit langsamen Ausgleich	21
3.12	Beobachter mit variablen Gain	22
4.1	Drehzahlregler als Blackbox mit Eingangs- und Ausgangssignalen	23
4.2	Interne Struktur des PI-Drehzahlreglers für einen Motor	27
4.3	Positionsregler als Blackbox mit Eingangs- und Ausgangssignalen	28
4.4	Interne Struktur des Positionsreglers für die X-Achse	29
4.5	Abbildung des Gain-Scheduling-Algorithmus für den Positionsregler in Lateral- (konstanter P-Regler, grün) und Einfahrtsrichtung (PI-Regler, rot und blau).	32
4.6	Flowchart über die Anfahrkette des Positionsreglers mit Gain-Scheduling	34
4.7	Sollwert der Geschwindigkeit in der Vektorfahrweise über 200s X-Achse: Sollgeschwindigkeit in mm/s, Y-Achse: Zeit in s	36
4.8	Istwert der Geschwindigkeit in der Vektorfahrweise über 200s X-Achse: Istgeschwindigkeit in mm/s, Y-Achse: Zeit in s	36
4.9	Anfahrweise einer Station in Y-Richtung	37
4.10	Anfahrweise einer Station in X-Richtung	38
4.11	Schematischer Aufbau von Zugehörigkeitsfunktionen eines Fuzzy-Reglers(übernommen aus [7]).	41
4.12	Gesamtauswertung der Simulation mit Membership-Funktionen, Trajektorie, Winkelverlauf, Fehlern und Stellgrößen.	47

4.13 Vergleich der Start- und Endskalierungen für Positionsfehler, Fehler- änderung und Ausgangsgrößen.	48
4.14 Rekonstruierte Roboterfahrt in der XY-Ebene mit Start, Ende und Zielpunkten.	51
4.15 Rekonstruierte Roboterfahrt in der y-Richtung.	52
4.16 Rekonstruierte Roboterfahrt in der XY-Ebene mit Richtungspfeilen. .	53
4.17 Weiterer X-Verlauf einer aufgezeichneten Fahrt mit Zielniveaus und Toleranzhinweis.	54

Tabellenverzeichnis

1	Konventionen	v
2.1	Gewerk 2 → Gewerk 3	5
2.2	Gewerk 3 → Gewerk 2	5
2.3	Zustände der internen Statemachine	5
4.1	Parameter des Drehzahlreglers	27
4.2	Exemplarischer Ablauf des Drehzahlreglers in TwinCAT	27
4.3	Exemplarischer Ablauf des Positionsreglers in TwinCAT	29
4.4	Fortsetzung eines exemplarischen Ablaufs des Positionsreglers in Twin- CAT	30
4.5	Linguistische Zustände des Fuzzy-Reglers	40
4.6	Ein- und Ausgangsgrößen der drei Regelkanäle	41
4.7	Regelmatrix des Fuzzy-Reglers	42
4.8	Zentrale Konfiguration der Simulation	44
4.9	Bestandteile der Kostenfunktion	45
4.10	Vergleich der Startparameter und optimierten Skalierungsparameter .	47

Literatur

- [1] Prof. Dr. Michael Erhard. *Nichtlineare Regelung*. Vorlesungsunterlagen, Hochschule für Angewandte Wissenschaften, Wintersemester 2025. 2025.
- [2] Nacer Hacene und Boubekeur Mendil. „Motion Analysis and Control of Three-Wheeled Omnidirectional Mobile Robot“. In: *Journal of Control, Automation and Electrical Systems* 30.2 (2019), S. 194–213. DOI: 10.1007/s40313-019-00439-0.
- [3] Jan Lunze. *Regelungstechnik 1: Systemtheoretische Grundlagen, Analyse und Entwurf einschleifiger Regelungen*. 10. Aufl. Springer Vieweg, 2016. ISBN: 978-3-642-29532-3.
- [4] Manel Mendili und Faouzi Bouani. „Predictive Control of Mobile Robot Using Kinematic and Dynamic Models“. In: *Journal of Control Science and Engineering* 2017 (2017), S. 5341381. DOI: 10.1155/2017/5341381.
- [5] Kaustav Mondal u. a. „Comparison of Kinematic and Dynamic Model Based Linear Model Predictive Control of Non-Holonomic Robot for Trajectory Tracking: Critical Trade-offs Addressed“. In: *Proceedings of the IASTED International Conference on Mechatronics and Control (MC 2019)*. 2019, S. 9–17. DOI: 10.2316/P.2019.860-017.
- [6] Anh-Tu Nguyen und Cong-Thanh Vu. „Mobile Robot Motion Control Using a Combination of Fuzzy Logic Method and Kinematic Model“. In: *Intelligent Systems and Networks*. Bd. 471. Lecture Notes in Networks and Systems. Springer, 2022, S. 495–503. DOI: 10.1007/978-981-19-3394-3_56.
- [7] Ahmet Selim Pehlivan, Beste Bahceci und Kemalettin Erbatur. „Genetically Optimized Pitch Angle Controller of a Wind Turbine with Fuzzy Logic Design Approach“. In: *Energies* 15.18 (2022), S. 6705. DOI: 10.3390/en15186705.